

Constituency Parsing

The goal of constituency parsing is to uncover the hierarchical structure of a sentence. Instead of seeing a sentence as just a flat list of words, parsing reveals its "skeleton."

Motivation

- Constituency parsing is motivated by the need to reveal and represent the hierarchical syntactic structure of sentences in natural language processing.
- It breaks down a sentence into sub-phrases or constituents such as noun phrases (NP), verb phrases (VP), and so on, which reflect how words group together to form meaningful units.
- This structure is critical for understanding grammatical relations and for various downstream applications like grammar checking, semantic analysis, and question answering.

Constituency

- Syntactic constituency is the idea that groups of words can behave as single units, or constituents.
- Part of developing a grammar involves building an inventory of the constituents in the language.
- How do words group together in English? Consider the noun phrase, a sequence of words surrounding at least one noun.
- Here are some examples of noun phrases
 - Harry the Horse
 - the Broadway coppers
 - three parties from Brooklyn

- What evidence do we have that these words group together (or “form constituents”)?
- One piece of evidence is that they can all appear in similar syntactic environments, for example, before a verb.
 - Three parties from Brooklyn arrive. . .
 - Harry the horse attracts. . .
 - The Broadway coppers love. . .
- However, this is not true of each of the individual words that make up a noun phrase
 - from arrive . . .
 - the is. . .

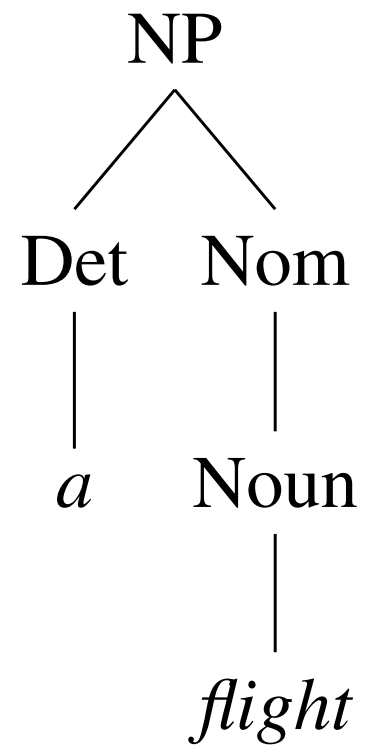
Context-Free Grammars

- A widely used formal system for modelling constituent structure in natural language is the context-free grammar, or CFG.
- A context-free grammar consists of a set of rules or productions, each of which expresses the ways that symbols of the language can be grouped and ordered together, and a lexicon of words and symbols.
- For example, the following productions express that an NP (or noun phrase) can be composed of either a ProperNoun or a determiner (Det) followed by a Nominal; a Nominal in turn can consist of one or more Nouns.
 - $NP \rightarrow \text{ProperNoun}$
 - $NP \rightarrow \text{Det Nominal}$
 - $\text{Nominal} \rightarrow \text{Noun} \mid \text{Nominal Noun}$

- Context-free rules can be hierarchically embedded, so we can combine the previous rules with others, like the following, that express facts about the lexicon:
 - $\text{Det} \rightarrow a$
 - $\text{Det} \rightarrow \text{the}$
 - $\text{Noun} \rightarrow \text{flight}$
- The symbols that are used in a CFG are divided into two classes.
- The symbols that correspond to words in the language (“the”, “nightclub”) are called **terminal symbols**.
- The **symbols that express abstractions over these terminals are called non-terminals**.

- In each context-free rule, the item to the right of the arrow ($\rightarrow$) is an ordered list of one or more terminals and non-terminals symbol expressing some cluster or generalization.
- The non-terminal associated with each word in the lexicon is its lexical category, or part of speech.
- A CFG can be thought of in two ways:
 - as a device for generating sentences
 - and as a device for assigning a structure to a given sentence.
- Viewing a CFG as a generator, we can read the $\rightarrow$ arrow as “rewrite the symbol on the left with the string of symbols on the right”

- So starting from the symbol: NP
- we can use our first rule to rewrite NP as: Det Nominal
- and then rewrite Nominal as: Noun
- and finally rewrite these parts-of-speech as: a flight
- We say the string “a flight” can be derived from the non-terminal NP.
- Thus, a CFG can be used to generate a set of strings.
- This sequence of rule expansions is called a **derivation** of the string of words.
- It is common to represent a derivation by a parse tree



A parse tree for “a flight”

- The formal language defined by a CFG is the set of strings that are derivable from the designated start symbol.
- Each grammar must have one designated start symbol, which is often called **S**.
- Since context-free grammars are often used to define sentences, S is usually interpreted as the “sentence” node

- Let's add a few additional rules to our inventory. The following rule expresses the fact that a sentence can consist of a noun phrase followed by a verb phrase

$S \rightarrow NP VP$ I prefer a morning flight

- A verb phrase in English consists of a verb followed by assorted other things;
- for example, one kind of verb phrase consists of a verb followed by a noun phrase:

$VP \rightarrow Verb NP$ prefer a morning flight

- Or the verb may be followed by a noun phrase and a prepositional phrase:

VP → Verb NP PP leave Boston in the morning

- Or the verb phrase may have a verb followed by a prepositional phrase alone:

VP → Verb PP leaving on Thursday

- A prepositional phrase generally has a preposition followed by a noun phrase.
- For example, a common type of prepositional phrase in the ATIS corpus is used to indicate location or direction:

PP → Preposition NP from Los Angeles

Sample Lexicon say L_0

A lexicon is essentially the "dictionary" of the grammar. It maps Non-terminals (parts of speech) to Terminals (actual words).

Noun → flights | flight | breeze | trip | morning

Verb → is | prefer | like | need | want | fly | do

Adjective → cheapest | non-stop | first | latest
| other | direct

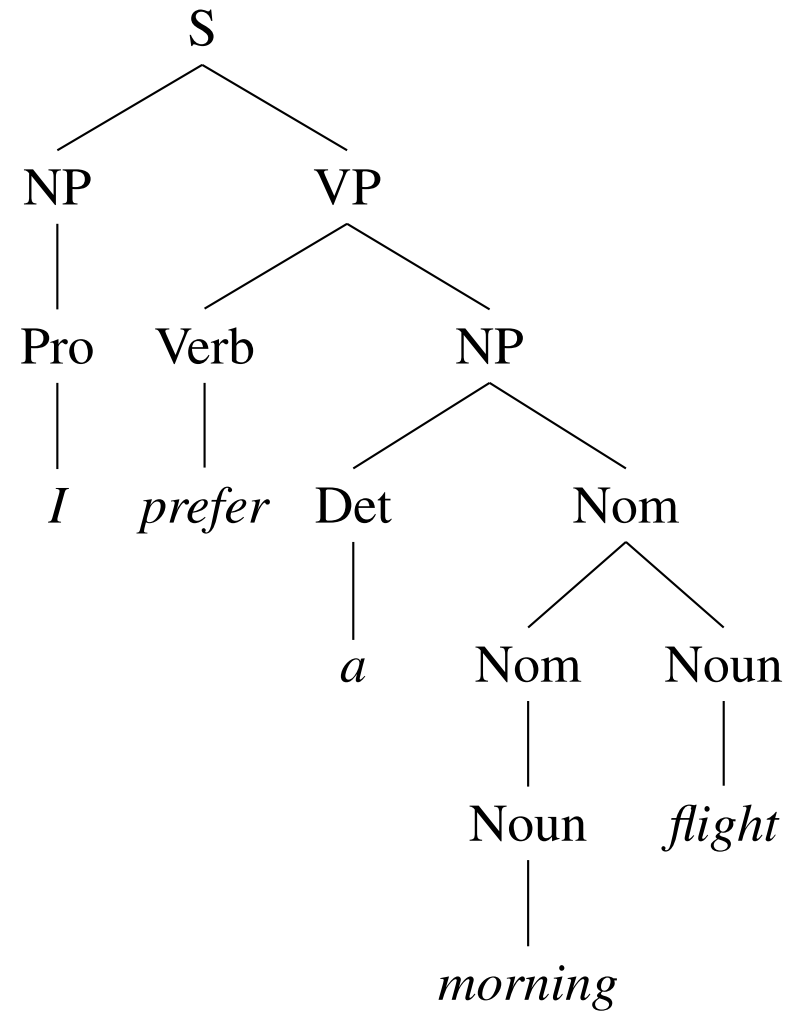
Pronoun → me | I | you | it

Proper-Noun → Alaska | Baltimore | Los Angeles
| Chicago | United | American

Determiner → the | a | an | this | these | that

Preposition → from | to | on | near | in

The parse tree for “I prefer a morning flight”
according to grammar L0



While trees are visually intuitive, they take up a lot of space. Computationally, it is often easier to represent this structure using bracketed notation.

- We can also represent a parse tree in a more compact format called bracketed notation; here is the bracketed representation of the parse tree
- $[_S [_{NP} [_{Pro} I]] [_{VP} [_V \text{prefer}] [_{NP} [_{Det} a] [_{Nom} [_N \text{morning}] [_{Nom} [_N \text{flight}]]]]]]]$
- A CFG like that of L0 defines a formal language.
- Sentences (strings of words) that can be derived by a grammar are in the formal language defined by that grammar, and are called grammatical sentences.
- Sentences that cannot be derived by a given formal grammar are not in the language defined by that grammar and are referred to as ungrammatical.

A formal system that "generates" all and only the strings that are considered grammatical in a language.

- In linguistics, the use of formal languages to model natural languages is called **generative grammar** since the language is defined by the set of possible sentences “generated” by the grammar.

Formal Definition of Context-Free Grammar

- A context-free grammar G is defined by four parameters: N, Σ, R, S
- N : a set of non-terminal symbols (or variables)
- Σ : a set of terminal symbols (disjoint from N)
- R : a set of rules or productions, each of the form $A \rightarrow \beta$, where A is a non-terminal, β is a string of symbols from the infinite set of strings $(\Sigma \cup N)^*$
- S : a designated start symbol and a member of N

Convention

- | | |
|---|--|
| • Capital letters like A, B, and S | Non-terminals |
| • S | The start symbol |
| • Lower-case Greek letters like α , β , and γ | Strings drawn from $(\Sigma \cup N)^*$ |
| • Lower-case Roman letters like u, v, and w | Strings of terminals |

Derivation

- if $A \rightarrow \beta$ is a production of R and α and γ are any strings in the set $(\Sigma \cup N)^*$, then we say that $\alpha A \gamma$ directly derives $\alpha \beta \gamma$, or $\alpha A \gamma \Rightarrow \alpha \beta \gamma$.

- Derivation is then a generalization of direct derivation:

- Let $\alpha_1, \alpha_2, \dots, \alpha_m$ be strings in $(\Sigma \cup N)^*$, $m \geq 1$, such that

$$\alpha_1 \Rightarrow \alpha_2, \alpha_2 \Rightarrow \alpha_3, \dots, \alpha_{m-1} \Rightarrow \alpha_m$$

- We say that α_1 derives α_m , or $\alpha_1 \xRightarrow{*} \alpha_m$.
- We can then formally define the language L_G generated by a grammar G as the set of strings composed of terminal symbols that can be derived from the designated start symbol S .

$$\mathcal{L}_G = \{w \mid w \text{ is in } \Sigma^* \text{ and } S \xRightarrow{*} w\}$$

Treebanks

A Treebank is a large corpus (collection of text) where every single sentence has been manually paired with a syntactic parse tree.

- A corpus in which every sentence is annotated with a parse tree is called a treebank.
- Treebanks play an important role in parsing as well as in linguistic investigations of syntactic phenomena.
- Treebanks are generally made by running a parser over each sentence and then having the resulting parse hand-corrected by human linguists.
- The sentences in a treebank implicitly constitute a grammar of the language.

Parses from the LDC Treebank3 for (a) Brown and (b) ATIS sentences.

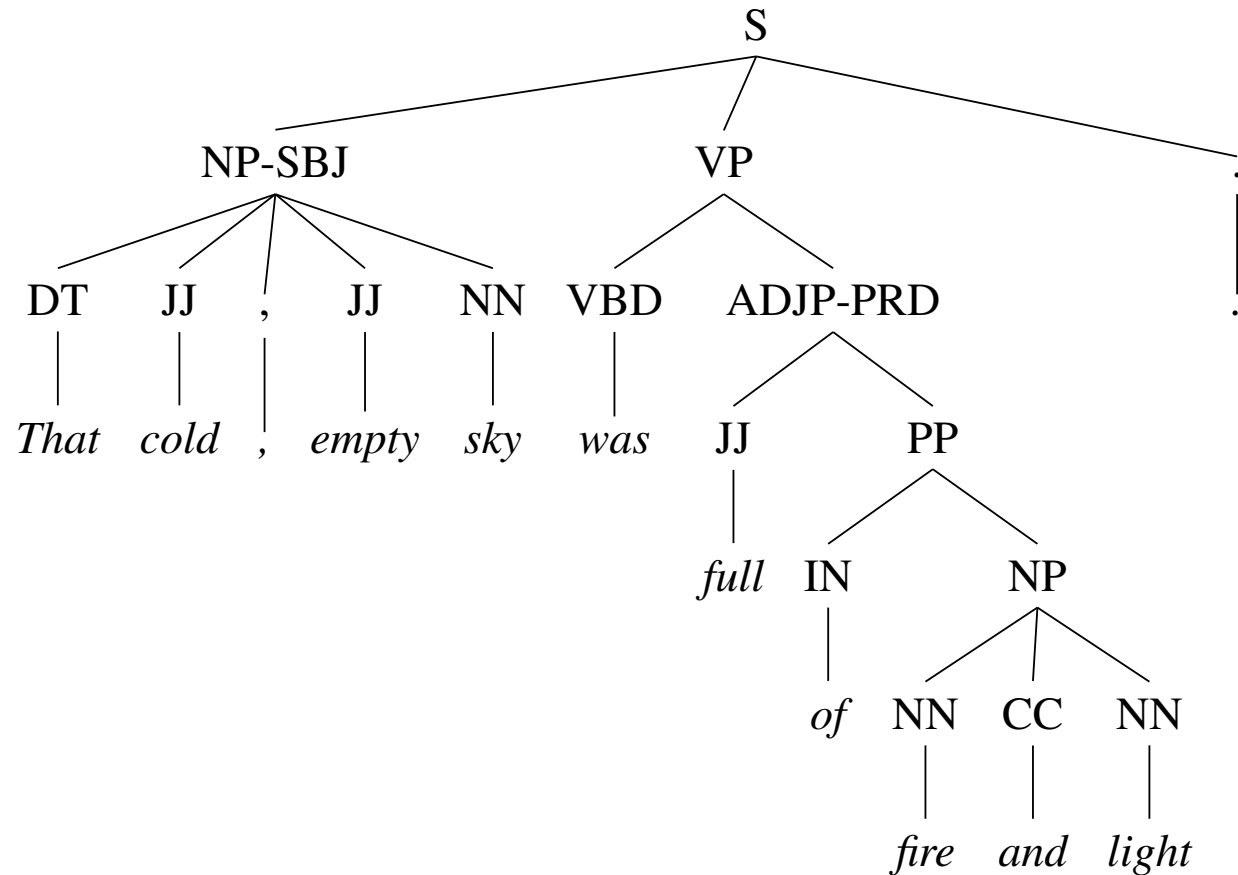
```
((S
  (NP-SBJ (DT That)
    (JJ cold) (, ,)
    (JJ empty) (NN sky) )
  (VP (VBD was)
    (ADJP-PRD (JJ full)
      (PP (IN of)
        (NP (NN fire)
          (CC and)
          (NN light) ))))
  (. .) ))
```

(a)

```
((S
  (NP-SBJ The/DT flight/NN )
  (VP should/MD
    (VP arrive/VB
      (PP-TMP at/IN
        (NP eleven/CD a.m/RB ))
      (NP-TMP tomorrow/NN )))))
```

(b)

The tree corresponding to the Brown corpus sentence in the previous figure.



Grammar Equivalence and Normal Form

- We say that two grammars are strongly equivalent if they generate the same set of strings and if they assign the same phrase structure to each sentence (allowing merely for renaming of the non-terminal symbols).
- Two grammars are weakly equivalent if they generate the same set of strings but do not assign the same phrase structure to each sentence.

Strong Equivalence: Two grammars are strongly equivalent if they generate the same set of strings and assign the same phrase structure (trees) to those strings.

Weak Equivalence: Two grammars are weakly equivalent if they generate the same set of strings but assign different phrase structures to them.

Grammar Equivalence and Normal Form

Chomsky Normal Form (CNF): In computational linguistics, it is often easier to work with grammars that follow a strict, standardized format. The most common is Chomsky Normal Form.

- It is sometimes useful to have a normal form for grammars, in which each of the productions takes a particular form.
- For example, a context-free grammar is in Chomsky normal form (Chomsky, 1963) if it is ϵ -free and if in addition each production is either of the form $A \rightarrow BC$ or $A \rightarrow a$.
- That is, the right-hand side of each rule either has two non-terminal symbols or one terminal symbol.
- Chomsky normal form grammars are binary branching, that is they have binary trees (down to the prelexical nodes).

The Rules:

Every production must take one of two forms: $A \rightarrow BC$ (A non-terminal expands to exactly two non-terminals).

$A \rightarrow a$ (A non-terminal expands to exactly one terminal/word).

- Any context-free grammar can be converted into a weakly equivalent Chomsky normal form grammar.
- For example, a rule of the form

$$A \rightarrow B C D$$

can be converted into the following two CNF rules

$$A \rightarrow B X$$

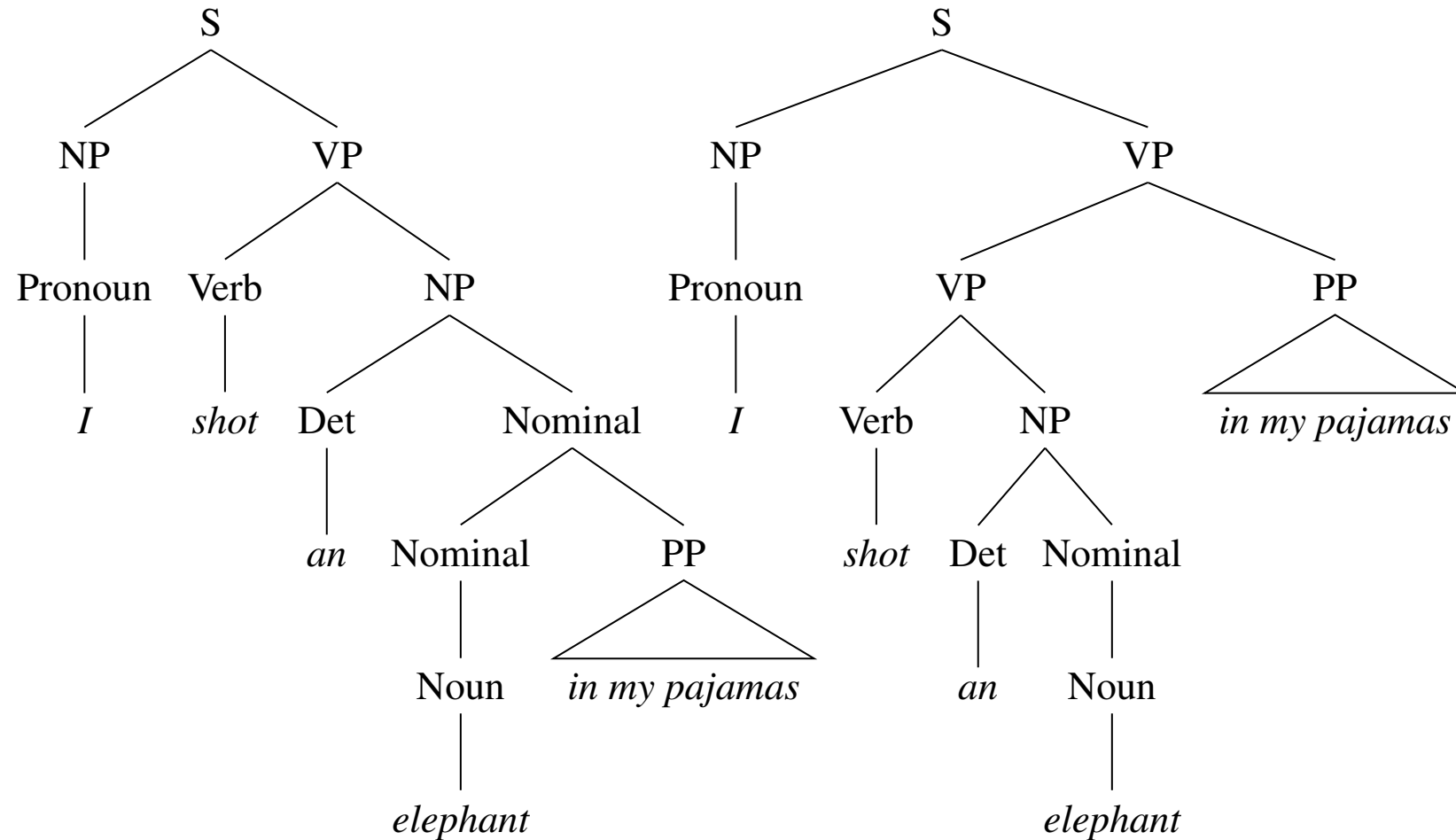
$$X \rightarrow C D$$

Ambiguity

- Ambiguity is the most serious problem faced by syntactic parsers.
- We have already seen part-of-speech ambiguity
- Here, we introduce a new kind of ambiguity, called structural ambiguity
- Structural ambiguity occurs when the grammar can assign more than one parse to a sentence.

Ambiguity occurs when a grammar can assign more than one valid parse tree to a single sentence. This is the "most serious problem" for syntactic parsers because the computer doesn't know which meaning was intended.

Two parse trees for an ambiguous sentence



- Structural ambiguity comes in many forms.
- Two common kinds of ambiguity are **attachment ambiguity** and **coordination ambiguity**.
- A sentence has an **attachment ambiguity** if a particular constituent can be attached to the parse tree at more than one place.
- Last parse tree shown has PP-attachment ambiguity: the preposition phrase can be attached either as part of the NP or as part of the VP.

- In **coordination ambiguity** phrases can be conjoined by a conjunction like and.
- For example, the phrase *old men and women* can be bracketed as [*old* [*men and women*]], referring to old men and old women, or as [*old men*] and [*women*] in which case it is only the men who are old.

It is unclear how conjunctions like "and" or "or" group words together (e.g., "old men and women" are the women also old?).

Parsing as search

- Parsing can be seen as search constrained by the grammar and the input
- Top down: start from S, find the derivations that lead to the sentence
- Bottom up: start from the sentence, find series of derivations (in reverse) that leads to S
- Search can be depth first or breadth first for both cases

Parsing as search: top down

S

she

saw a

duck

S → NP VP

S → Aux NP VP

NP → Det N

NP → Prn

NP → NP PP

VP → V NP

VP → V

VP → VP PP

PP → Prp NP

N → duck

N → park

V → duck

V → ducks

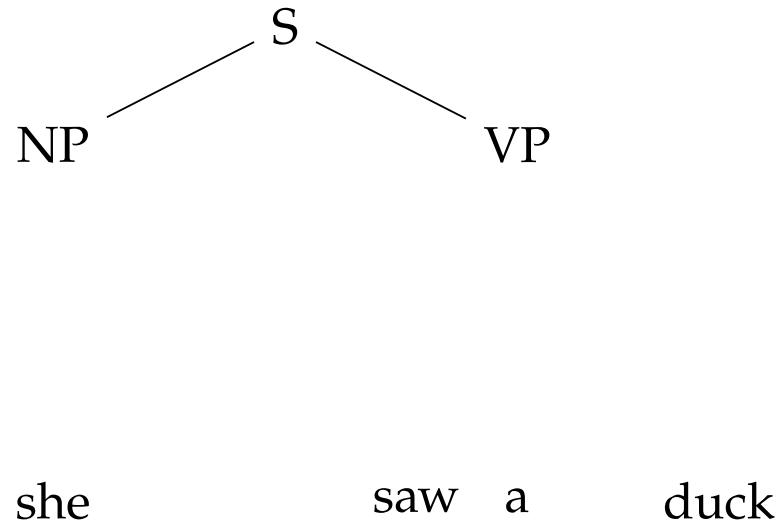
V → saw

Prn → she | her

Prp → in | with

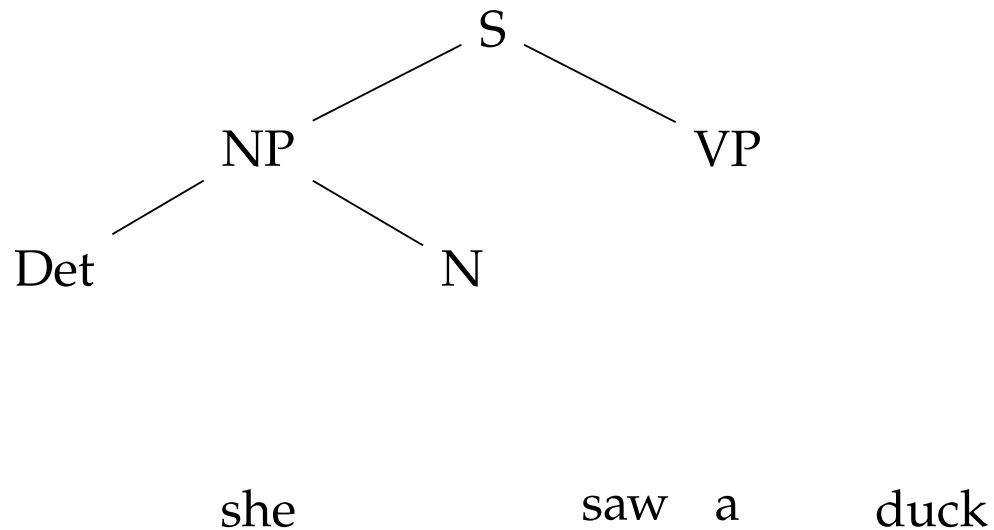
Det → a | the

Parsing as search: top down



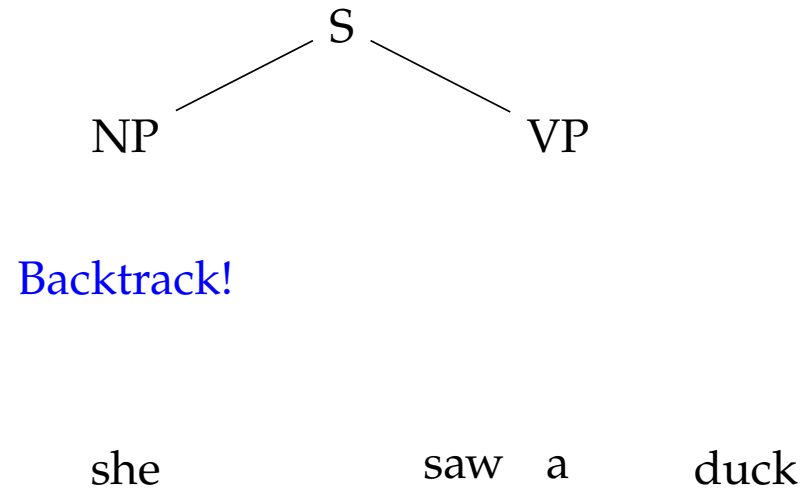
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: top down



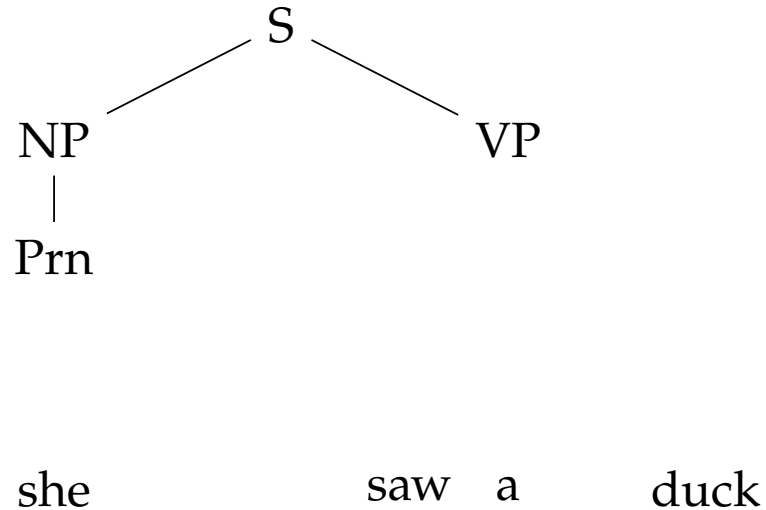
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: top down



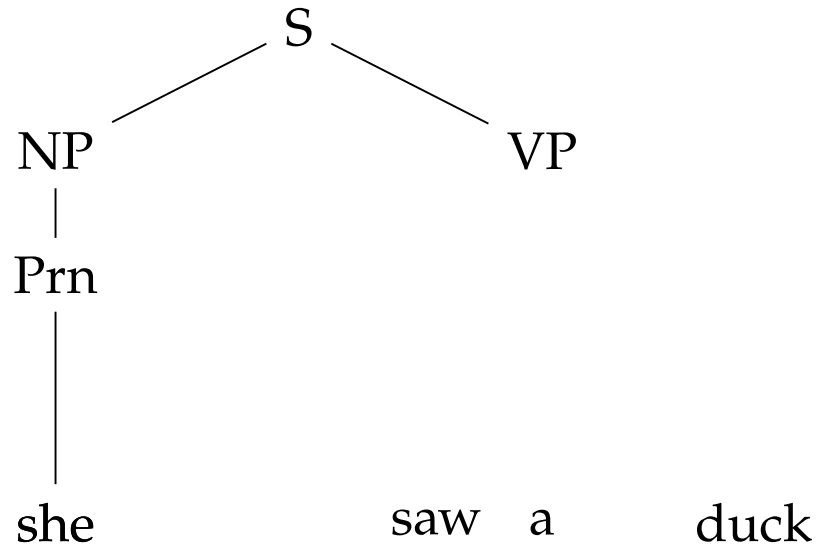
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: top down



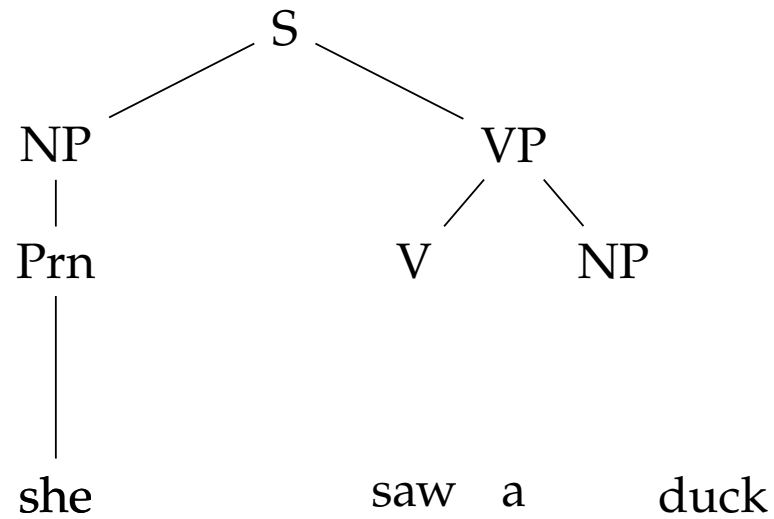
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: top down



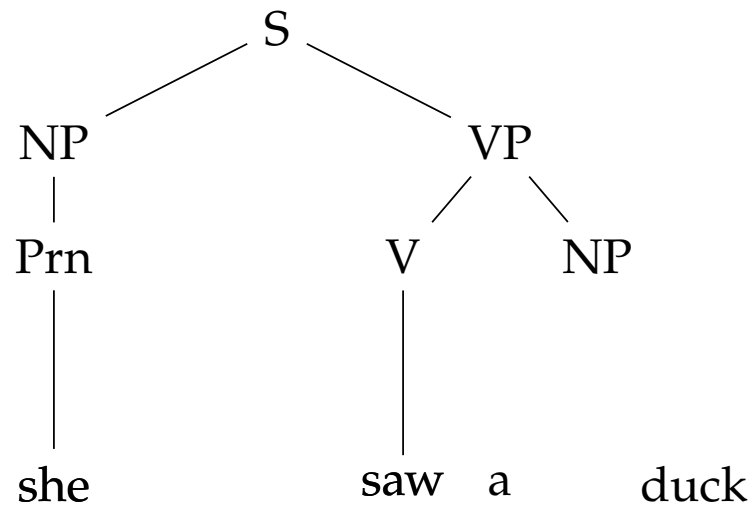
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: top down



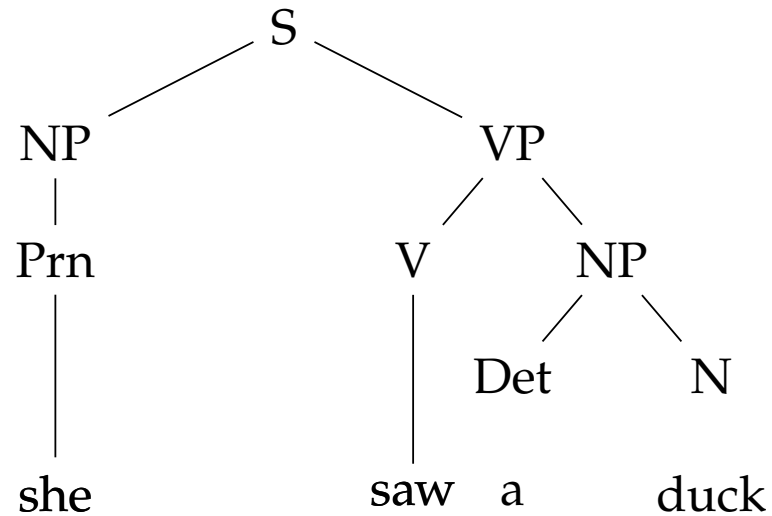
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: top down



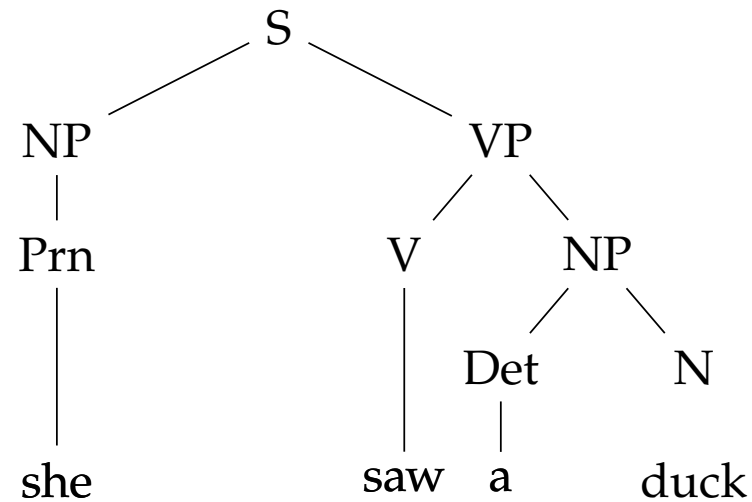
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: top down



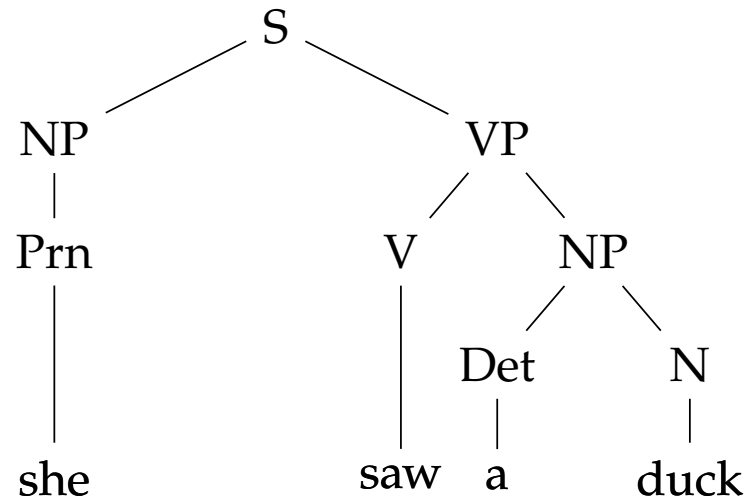
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: top down



S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: top down



S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: bottom up

S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

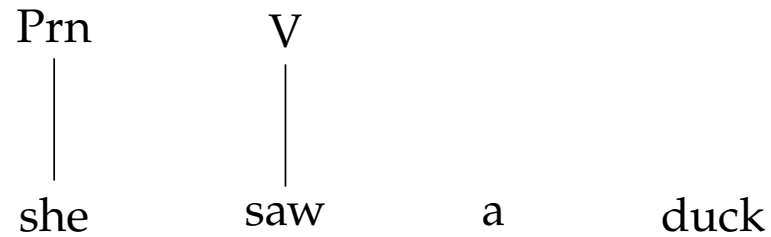
she saw a duck

Parsing as search: bottom up

Prn
|
she saw a duck

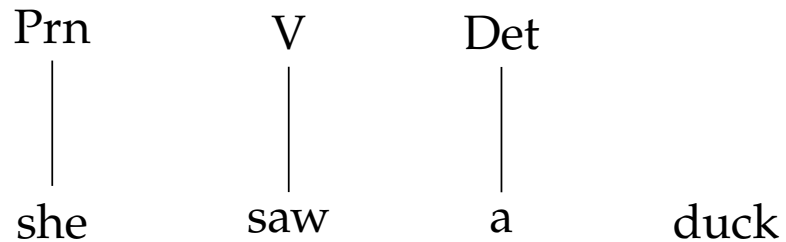
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: bottom up



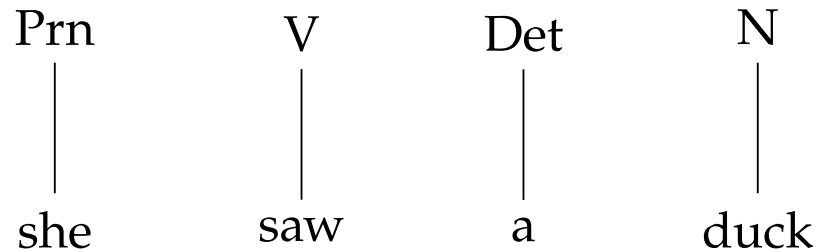
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: bottom up



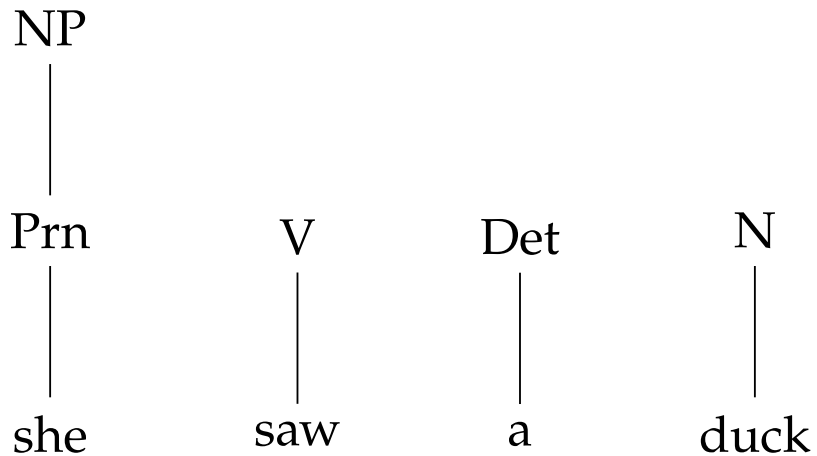
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: bottom up



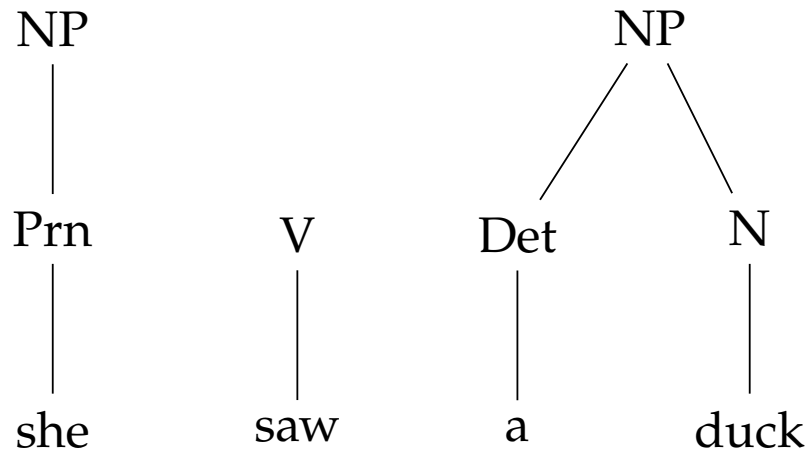
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: bottom up



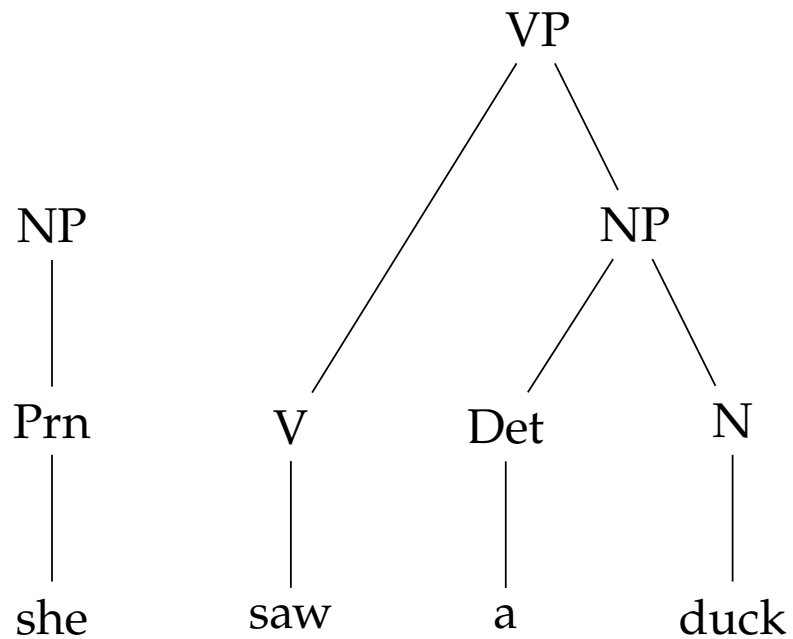
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: bottom up



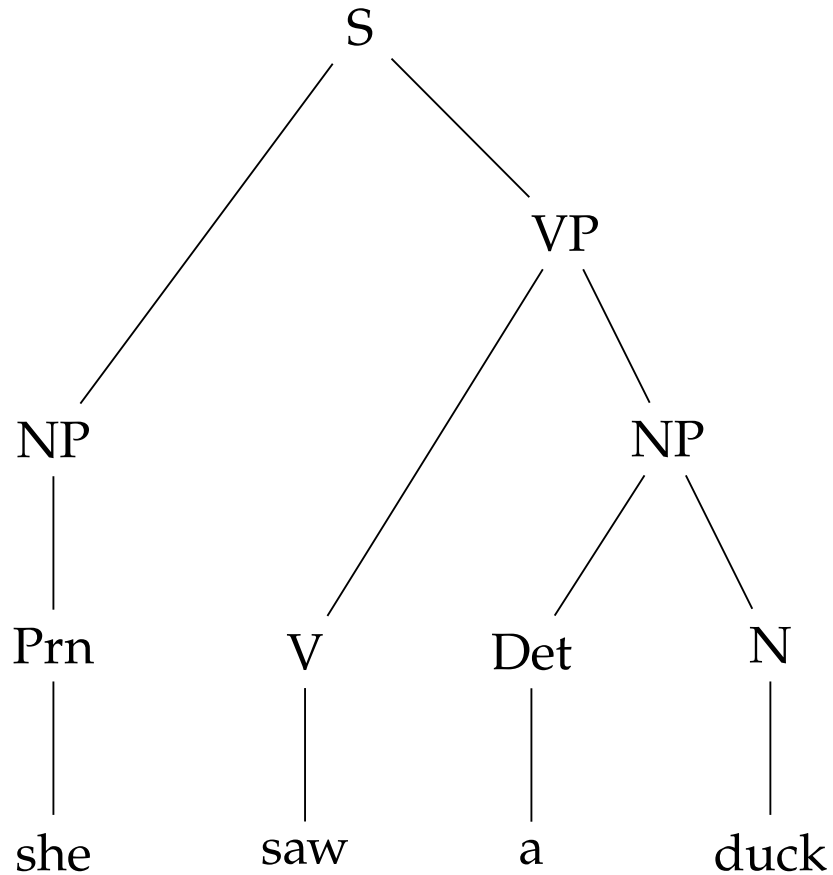
S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: bottom up



S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

Parsing as search: bottom up



S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

CKY Parsing: A Dynamic Programming Approach

- This section presents the Cocke-Kasami-Younger (CKY) algorithm, the most widely used dynamic-programming based approach to parsing.
- Dynamic programming provides a powerful framework for addressing the problems caused by ambiguity in grammars.
- Recall that a dynamic programming approach systematically fills in a table of solutions to subproblems.
- The complete table has the solution to all the subproblems needed to solve the problem as a whole.
- In the case of syntactic parsing, these subproblems represent parse trees for all the constituents detected in the input.

Conversion to Chomsky Normal Form

The CKY algorithm only works if the grammar is in CNF

- The CKY algorithm requires grammars to first be in Chomsky Normal Form (CNF).
- CNF are restricted to rules of the form $A \rightarrow BC$ or $A \rightarrow w$.
- That is, the right-hand side of each rule must expand either to two non-terminals or to a single terminal.
- Restricting a grammar to CNF does not lead to any loss in expressiveness, since any context-free grammar can be converted into a corresponding CNF grammar that accepts exactly the same set of strings as the original grammar.

Converting to CNF doesn't "weaken" the grammar; it just standardizes the format so the algorithm can consistently look for two-part splits.

- Rules with right-hand sides longer than 2 are normalized through the introduction of new non-terminals that spread the longer sequences over several new rules.
- Formally, if we have a rule like
- $A \rightarrow B C \gamma$
- we replace the leftmost pair of non-terminals with a new non-terminal and introduce a new production, resulting in the following new rules:
- $A \rightarrow X_1 \gamma$
- $X_1 \rightarrow B C$

$\mathcal{L}_1$ Grammar	$\mathcal{L}_1$ in CNF
$S \rightarrow NP VP$	$S \rightarrow NP VP$
$S \rightarrow Aux NP VP$	$S \rightarrow X1 VP$
	$X1 \rightarrow Aux NP$
$S \rightarrow VP$	$S \rightarrow book \mid include \mid prefer$
	$S \rightarrow Verb NP$
	$S \rightarrow X2 PP$
	$S \rightarrow Verb PP$
	$S \rightarrow VP PP$
$NP \rightarrow Pronoun$	$NP \rightarrow I \mid she \mid me$
$NP \rightarrow Proper-Noun$	$NP \rightarrow TWA \mid Houston$
$NP \rightarrow Det Nominal$	$NP \rightarrow Det Nominal$
$Nominal \rightarrow Noun$	$Nominal \rightarrow book \mid flight \mid meal \mid money$
$Nominal \rightarrow Nominal Noun$	$Nominal \rightarrow Nominal Noun$
$Nominal \rightarrow Nominal PP$	$Nominal \rightarrow Nominal PP$
$VP \rightarrow Verb$	$VP \rightarrow book \mid include \mid prefer$
$VP \rightarrow Verb NP$	$VP \rightarrow Verb NP$
$VP \rightarrow Verb NP PP$	$VP \rightarrow X2 PP$
	$X2 \rightarrow Verb NP$
$VP \rightarrow Verb PP$	$VP \rightarrow Verb PP$
$VP \rightarrow VP PP$	$VP \rightarrow VP PP$
$PP \rightarrow Preposition NP$	$PP \rightarrow Preposition NP$

CKY Recognition

- With our grammar now in CNF, each non-terminal node above the part-of-speech level in a parse tree will have exactly two daughters.
- A two-dimensional matrix can be used to encode the structure of an entire tree.
- For a sentence of length n , we will work with the upper-triangular portion of an $(n+1) \times (n+1)$ matrix.
- Each cell $[i,j]$ in this matrix contains the set of non-terminals that represent all the constituents that span positions i through j of the input.

A cell in the matrix represents a constituent that spans from position i to position j .

- Since our indexing scheme begins with 0, it's natural to think of the indexes as pointing at the gaps between the input words (as in 0 Book 1 that 2 flight 3).
- These gaps are often called **fenceposts**, on the metaphor of the posts between segments of fencing.
- It follows then that the cell that represents the entire input resides in position $[0,n]$ in the matrix.
- Since each non-terminal entry in our table has two daughters in the parse, it follows that for each constituent represented by an entry $[i,j]$, there must be a position in the input, k , where it can be split into two parts such that $i < k < j$.

- Given such a position k , the first constituent $[i,k]$ must lie to the left of entry $[i,j]$ somewhere along row i , and the second entry $[k,j]$ must lie beneath it, along column j .

CKY Steps:

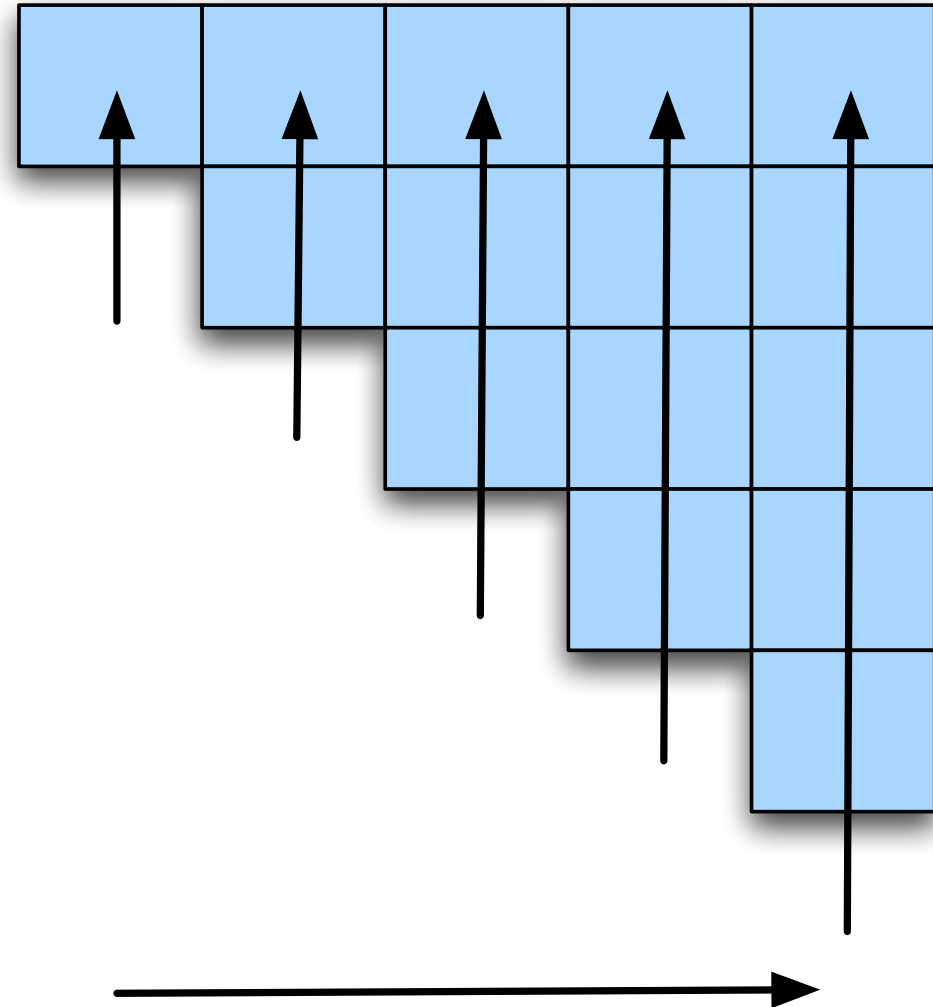
1. Initialize: Fill the diagonal cells $[i, i+1]$ with the parts of speech for each word.
2. Iterate: Fill the matrix column by column, bottom to top.
3. Combine: For each cell, check all possible split points k to see if existing constituents can form a larger one.
4. Terminate: Check if S is in the top-right cell.

CKY Algorithm

- The superdiagonal row in the matrix contains the parts of speech for each word in the input.
- The subsequent diagonals above that superdiagonal contain constituents that cover all the spans of increasing length in the input.
- Given this setup, CKY recognition consists of filling the parse table in the right way.
- To do this, we'll proceed in a bottom-up fashion so that at the point where we are filling any cell $[i,j]$, the cells containing the parts that could contribute to this entry (i.e., the cells to the left and the cells below) have already been filled.

An example: Book the flight through Houston

<i>Book</i>	<i>the</i>	<i>flight</i>	<i>through</i>	<i>Houston</i>
S, VP, Verb, Nominal, Noun [0,1]		S,VP,X2 [0,3]		S,VP,X2 [0,5]
	Det [1,2]	NP [1,3]		NP [1,5]
		Nominal, Noun [2,3]		Nominal [2,5]
			Prep [3,4]	PP [3,5]
				NP, Proper- Noun [4,5]



function CKY-PARSE(*words*, *grammar*) **returns** *table*

for $j \leftarrow$ **from** 1 **to** LENGTH(*words*) **do**

for all $\{A \mid A \rightarrow \text{words}[j] \in \text{grammar}\}$

$\text{table}[j-1, j] \leftarrow \text{table}[j-1, j] \cup A$

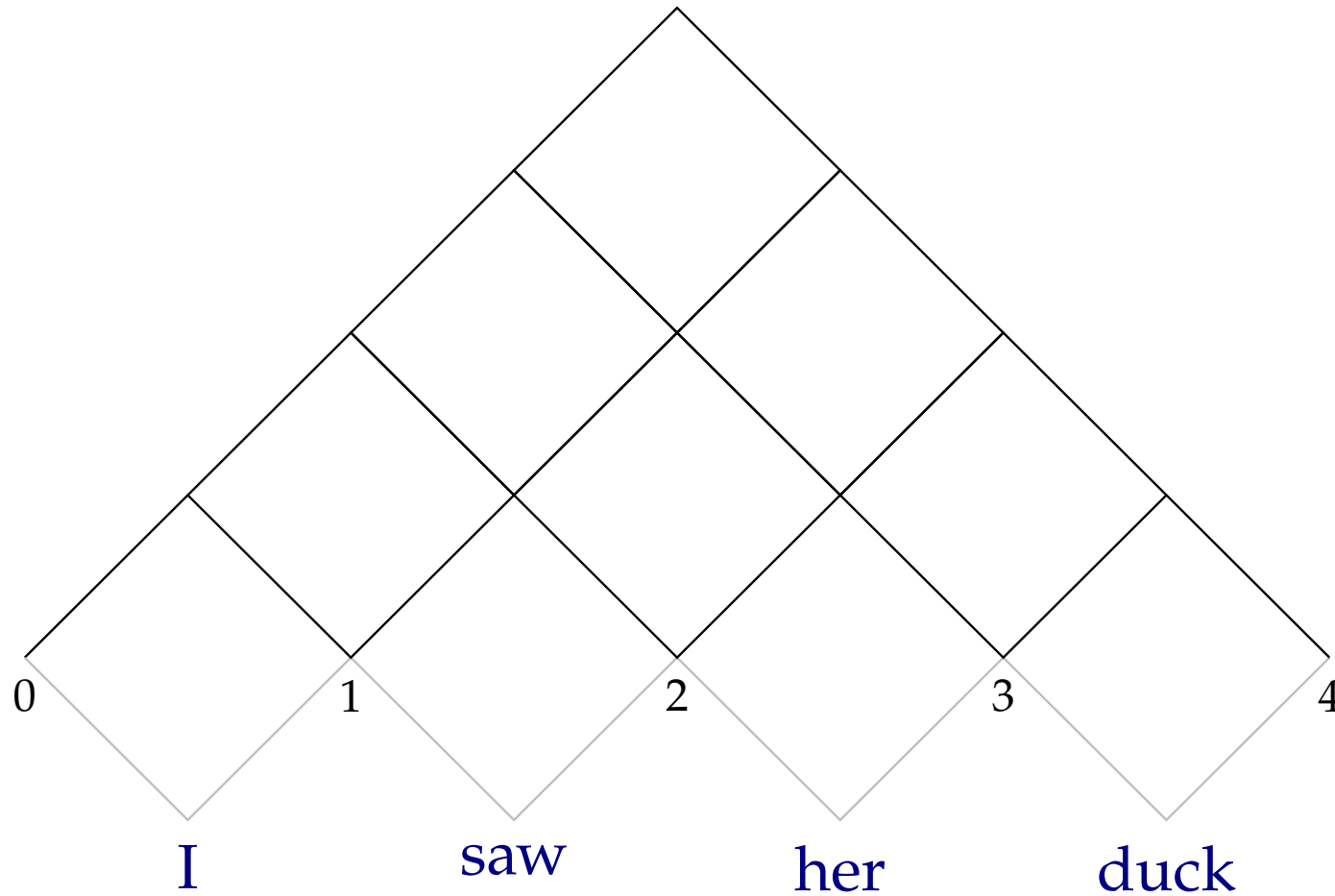
for $i \leftarrow$ **from** $j-2$ **down to** 0 **do**

for $k \leftarrow i+1$ **to** $j-1$ **do**

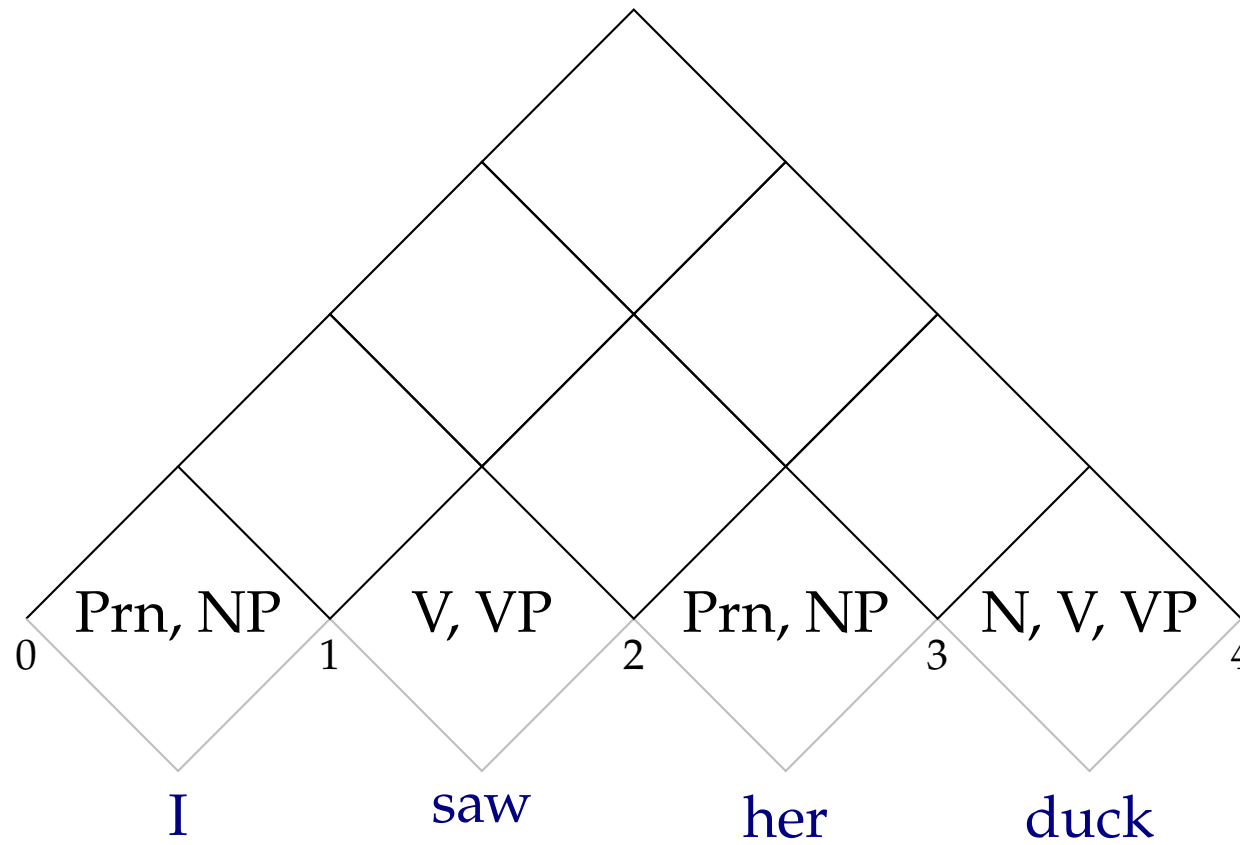
for all $\{A \mid A \rightarrow BC \in \text{grammar} \text{ and } B \in \text{table}[i, k] \text{ and } C \in \text{table}[k, j]\}$

$\text{table}[i, j] \leftarrow \text{table}[i, j] \cup A$

CKY demonstration

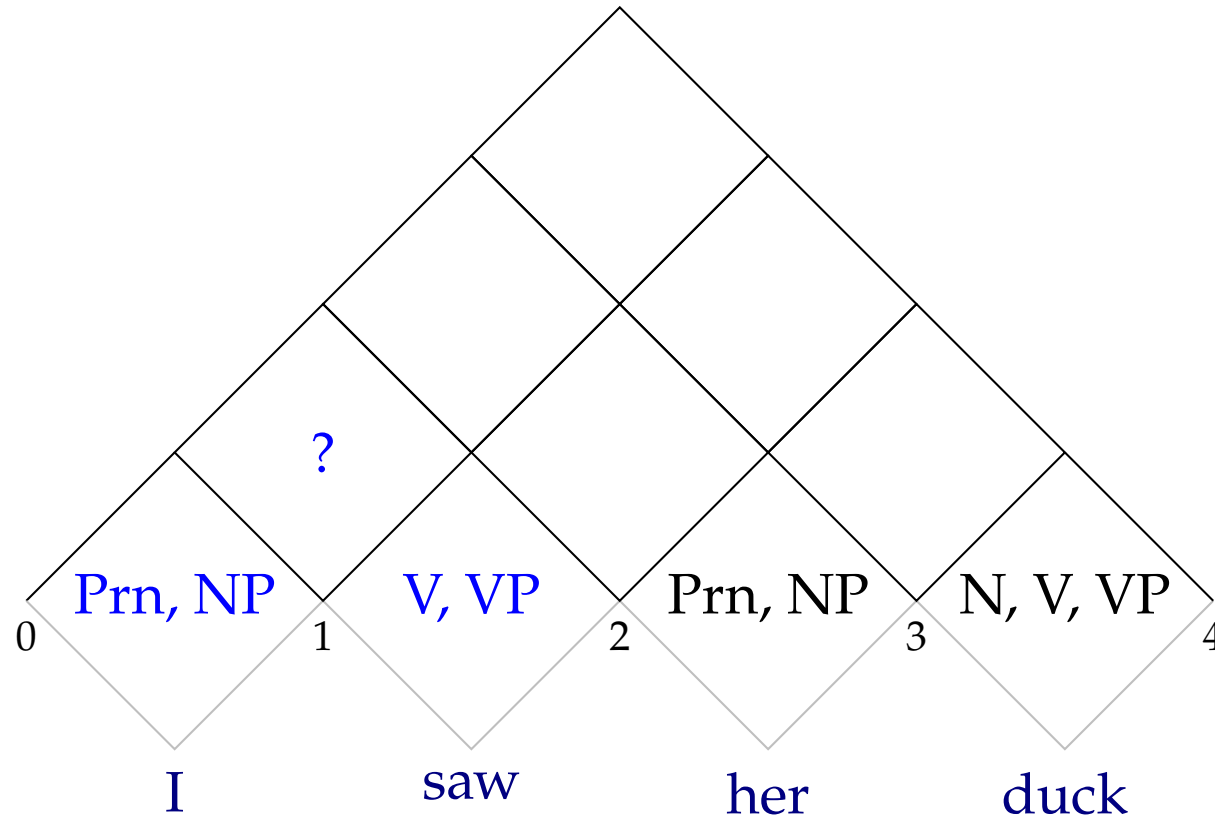


S $\rightarrow$ NP VP
S $\rightarrow$ Aux NP VP
NP $\rightarrow$ Det N
NP $\rightarrow$ Prn
NP $\rightarrow$ NP PP
VP $\rightarrow$ V NP
VP $\rightarrow$ V
VP $\rightarrow$ VP PP
PP $\rightarrow$ Prp NP
N $\rightarrow$ duck
N $\rightarrow$ park
V $\rightarrow$ duck
V $\rightarrow$ ducks
V $\rightarrow$ saw
Prn $\rightarrow$ she | her
Prp $\rightarrow$ in | with
Det $\rightarrow$ a | the



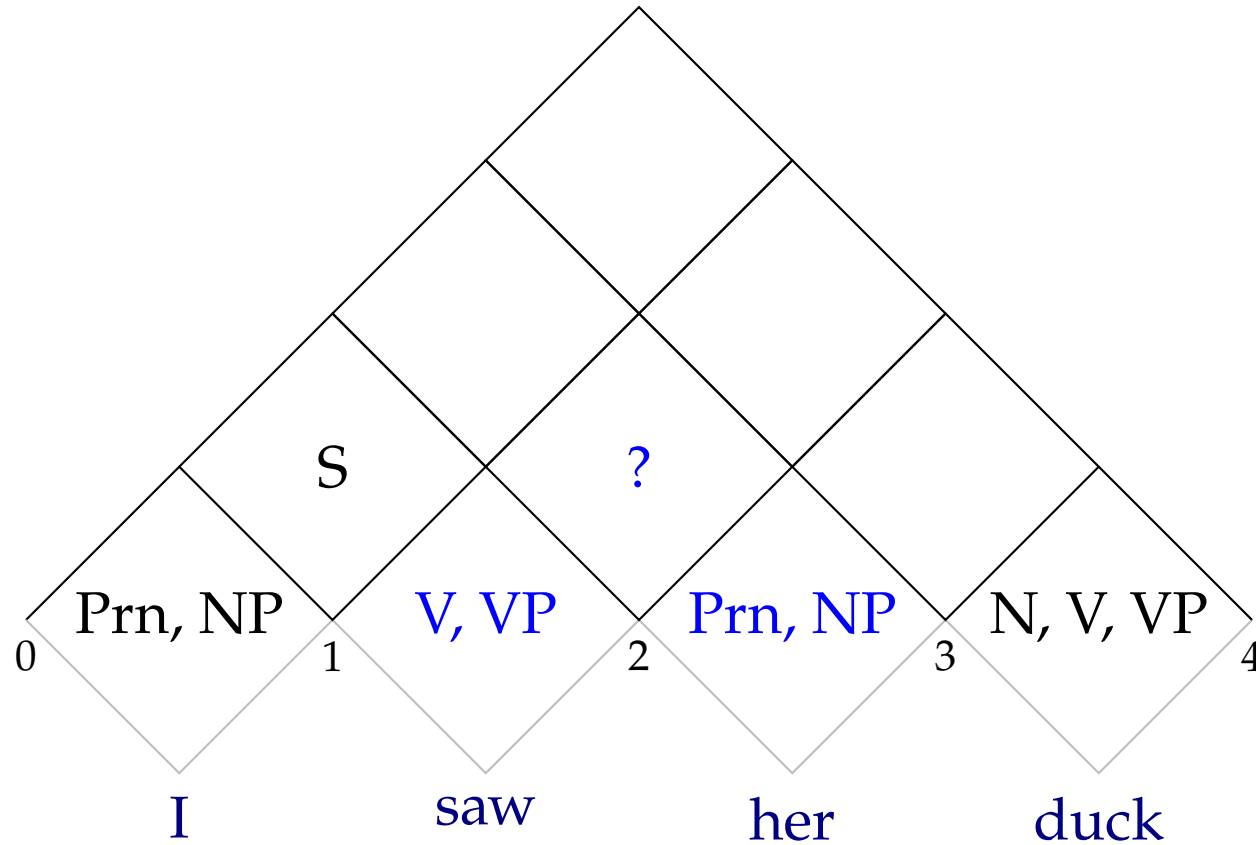
S → NP VP
 S → Aux NP VP
 NP → Det N
 NP → Prn
 NP → NP PP
 VP → V NP
 VP → V
 VP → VP PP
 PP → Prp NP
 N → duck
 N → park
 V → duck
 V → ducks
 V → saw
 Prn → she | her
 Prp → in | with
 Det → a | the

$S \rightarrow NP VP$

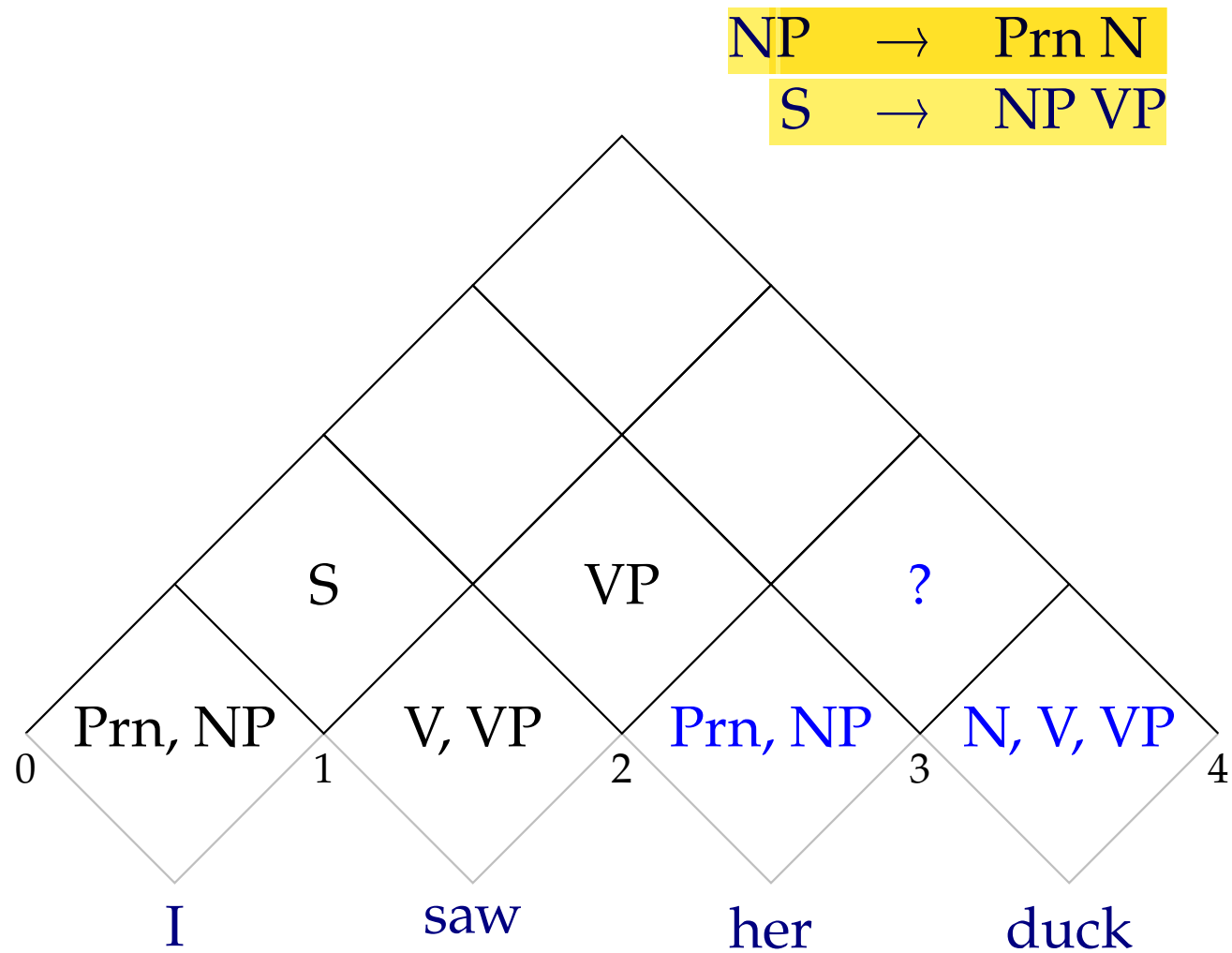


- $S \rightarrow NP VP$
- $S \rightarrow Aux NP VP$
- $NP \rightarrow Det N$
- $NP \rightarrow Prn$
- $NP \rightarrow NP PP$
- $VP \rightarrow V NP$
- $VP \rightarrow V$
- $VP \rightarrow VP PP$
- $PP \rightarrow Prp NP$
- $N \rightarrow duck$
- $N \rightarrow park$
- $V \rightarrow duck$
- $V \rightarrow ducks$
- $V \rightarrow saw$
- $Prn \rightarrow she \mid her$
- $Prp \rightarrow in \mid with$
- $Det \rightarrow a \mid the$

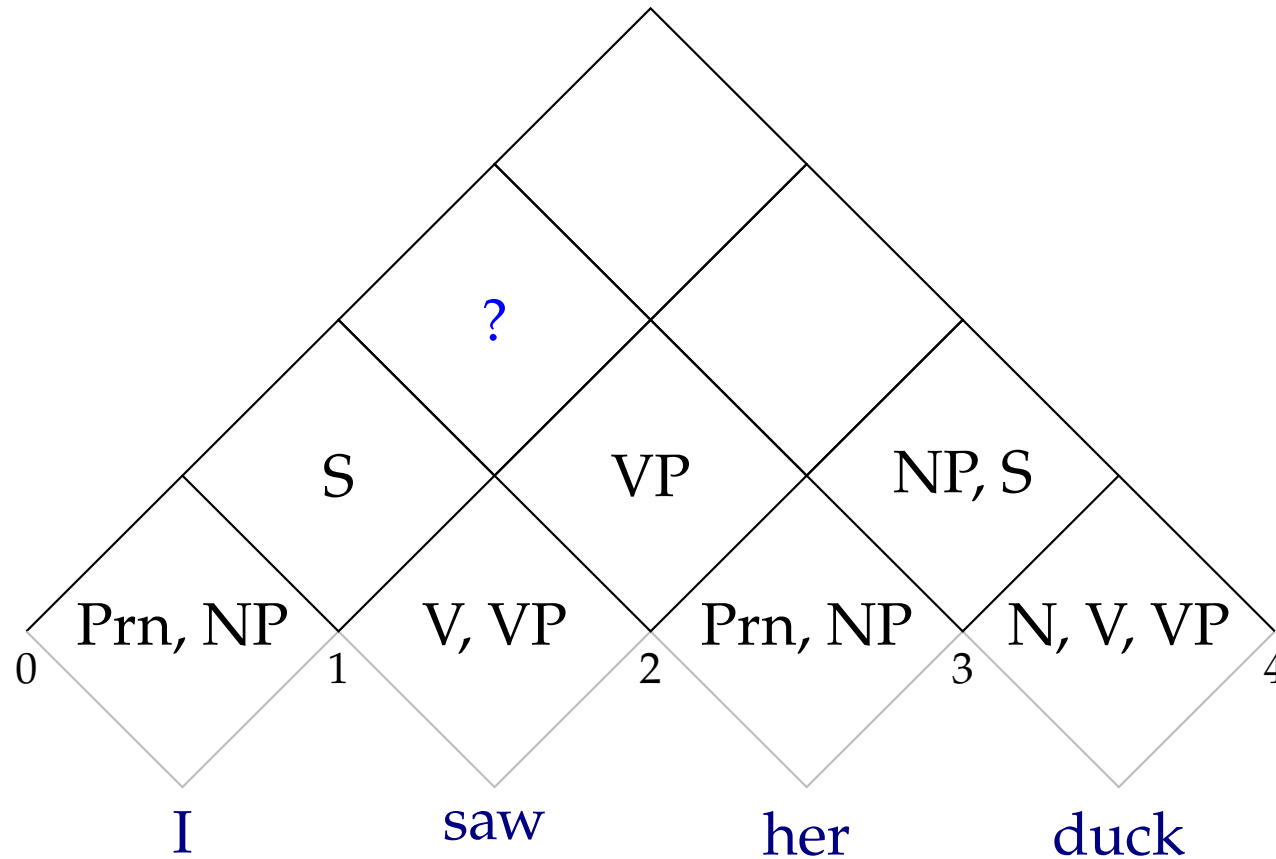
VP $\rightarrow$ V NP



S $\rightarrow$ NP VP
S $\rightarrow$ Aux NP VP
NP $\rightarrow$ Det N
NP $\rightarrow$ Prn
NP $\rightarrow$ NP PP
VP $\rightarrow$ V NP
VP $\rightarrow$ V
VP $\rightarrow$ VP PP
PP $\rightarrow$ Prp NP
N $\rightarrow$ duck
N $\rightarrow$ park
V $\rightarrow$ duck
V $\rightarrow$ ducks
V $\rightarrow$ saw
Prn $\rightarrow$ she | her
Prp $\rightarrow$ in | with
Det $\rightarrow$ a | the

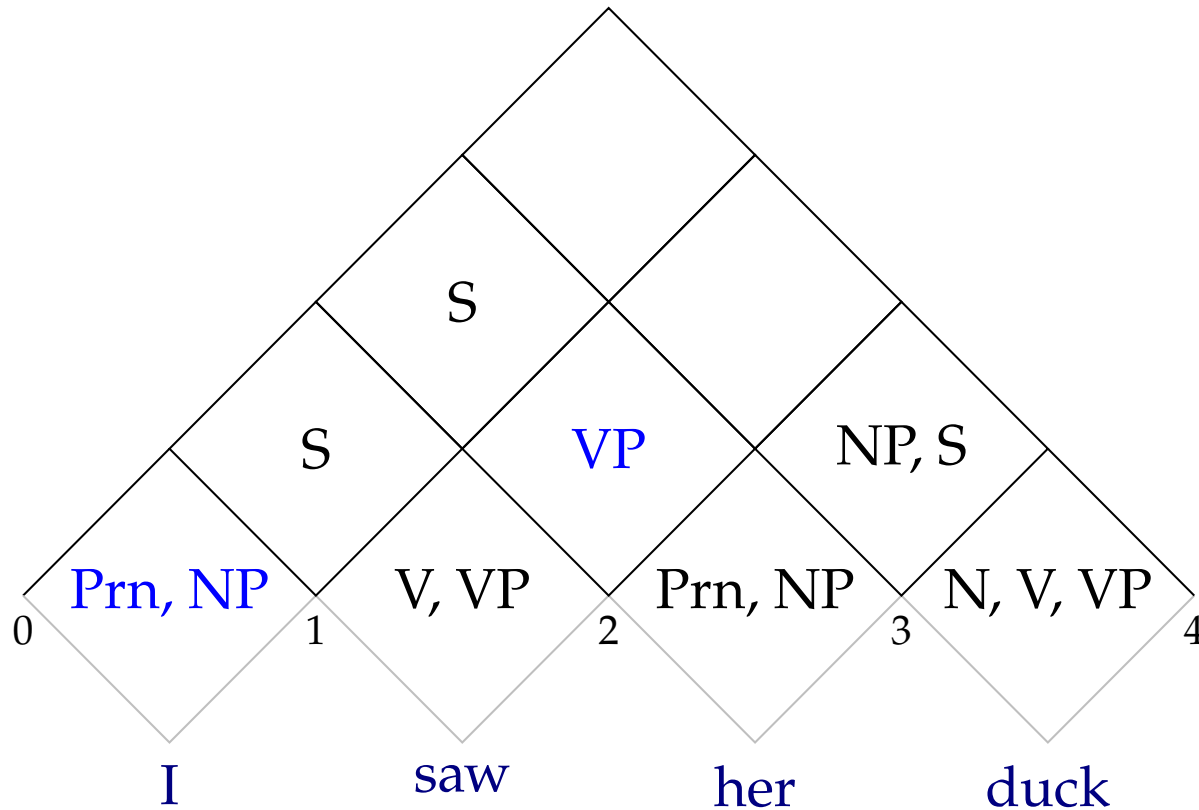


S → NP VP
S → Aux NP VP
NP → Det N
NP → Prn
NP → NP PP
VP → V NP
VP → V
VP → VP PP
PP → Prp NP
N → duck
N → park
V → duck
V → ducks
V → saw
Prn → she | her
Prp → in | with
Det → a | the

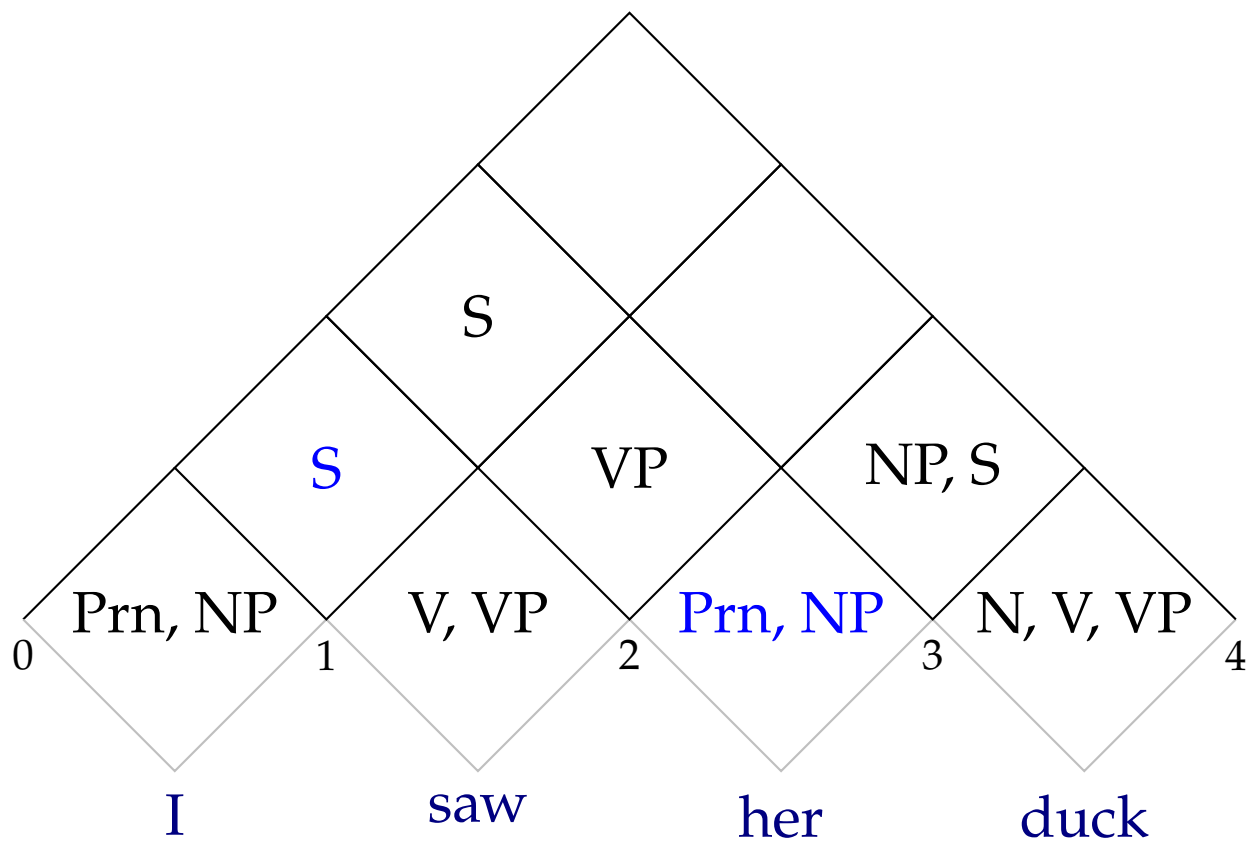


- S → NP VP
- S → Aux NP VP
- NP → Det N
- NP → Prn
- NP → NP PP
- VP → V NP
- VP → V
- VP → VP PP
- PP → Prp NP
- N → duck
- N → park
- V → duck
- V → ducks
- V → saw
- Prn → she | her
- Prp → in | with
- Det → a | the

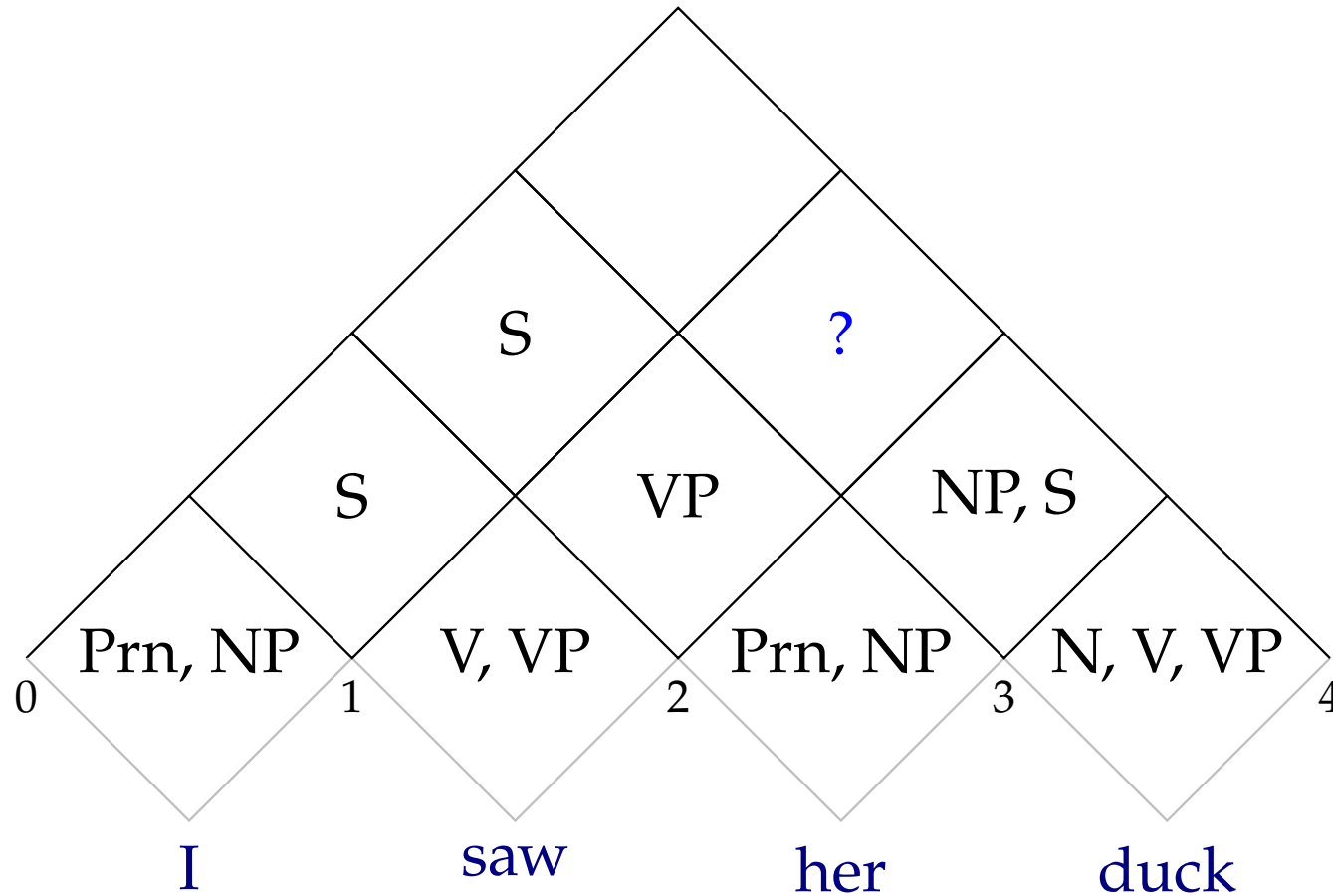
$S \rightarrow NP VP$



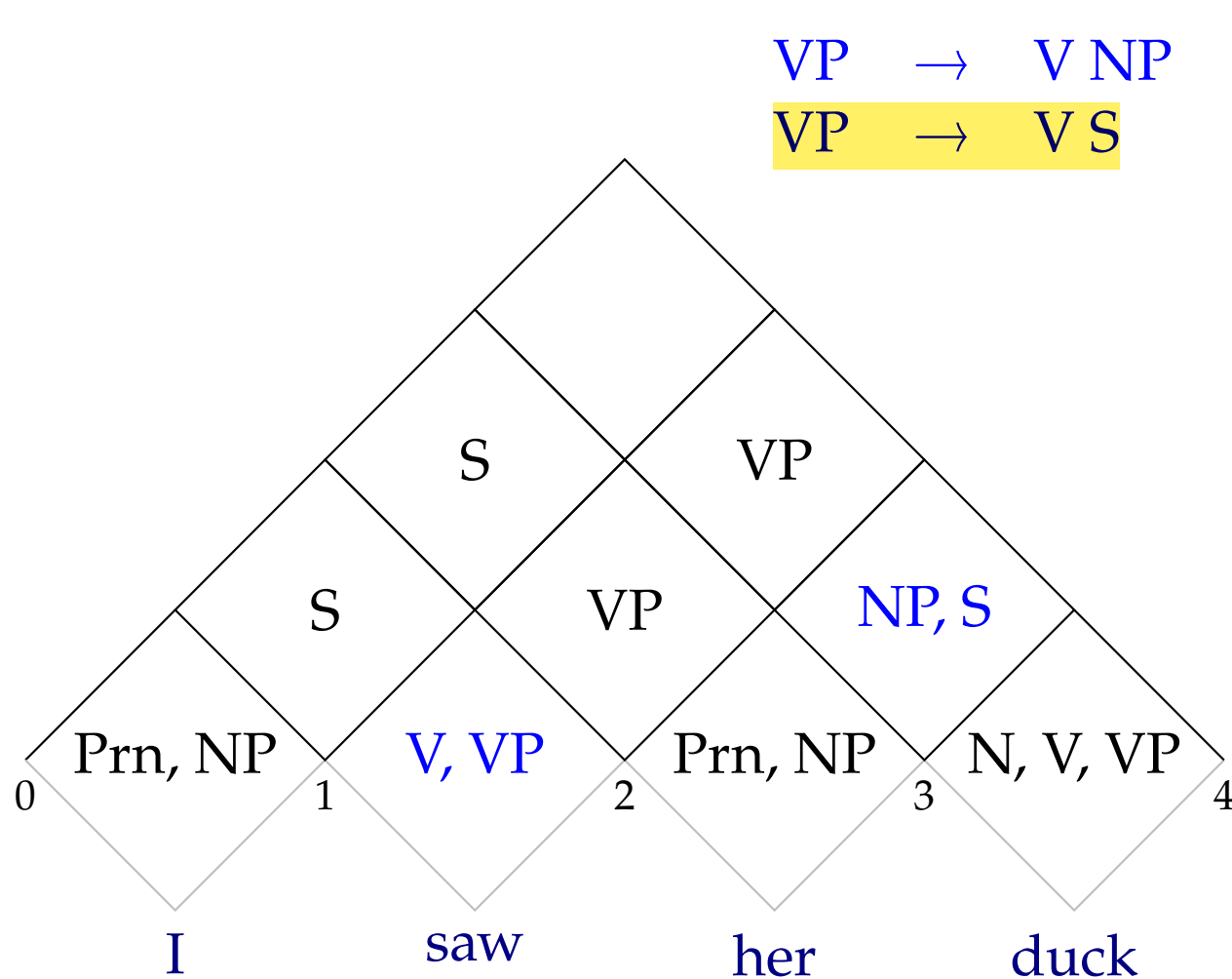
$S \rightarrow NP VP$
 $S \rightarrow Aux NP VP$
 $NP \rightarrow Det N$
 $NP \rightarrow Prn$
 $NP \rightarrow NP PP$
 $VP \rightarrow V NP$
 $VP \rightarrow V$
 $VP \rightarrow VP PP$
 $PP \rightarrow Prp NP$
 $N \rightarrow duck$
 $N \rightarrow park$
 $V \rightarrow duck$
 $V \rightarrow ducks$
 $V \rightarrow saw$
 $Prn \rightarrow she \mid her$
 $Prp \rightarrow in \mid with$
 $Det \rightarrow a \mid the$



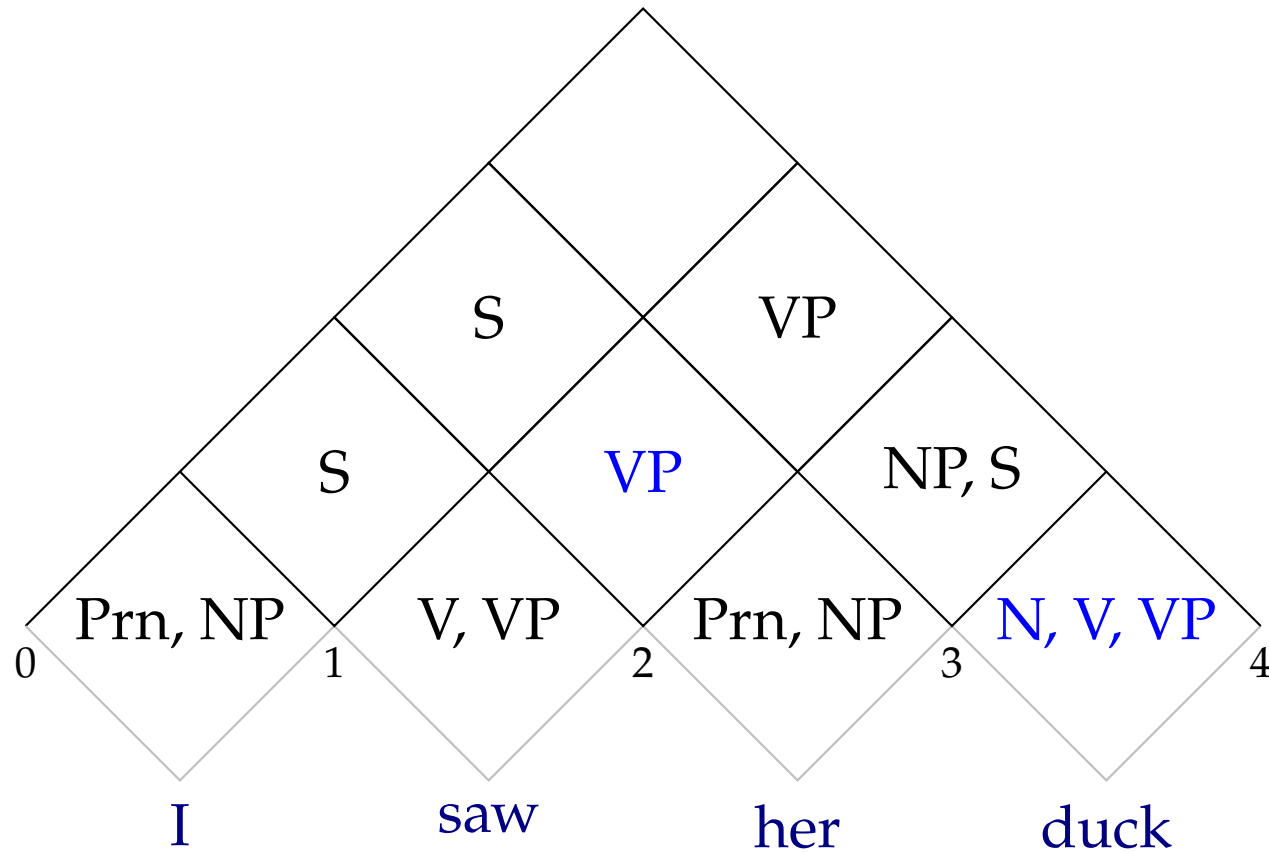
S → NP VP
 S → Aux NP VP
 NP → Det N
 NP → Prn
 NP → NP PP
 VP → V NP
 VP → V
 VP → VP PP
 PP → Prp NP
 N → duck
 N → park
 V → duck
 V → ducks
 V → saw
 Prn → she | her
 Prp → in | with
 Det → a | the



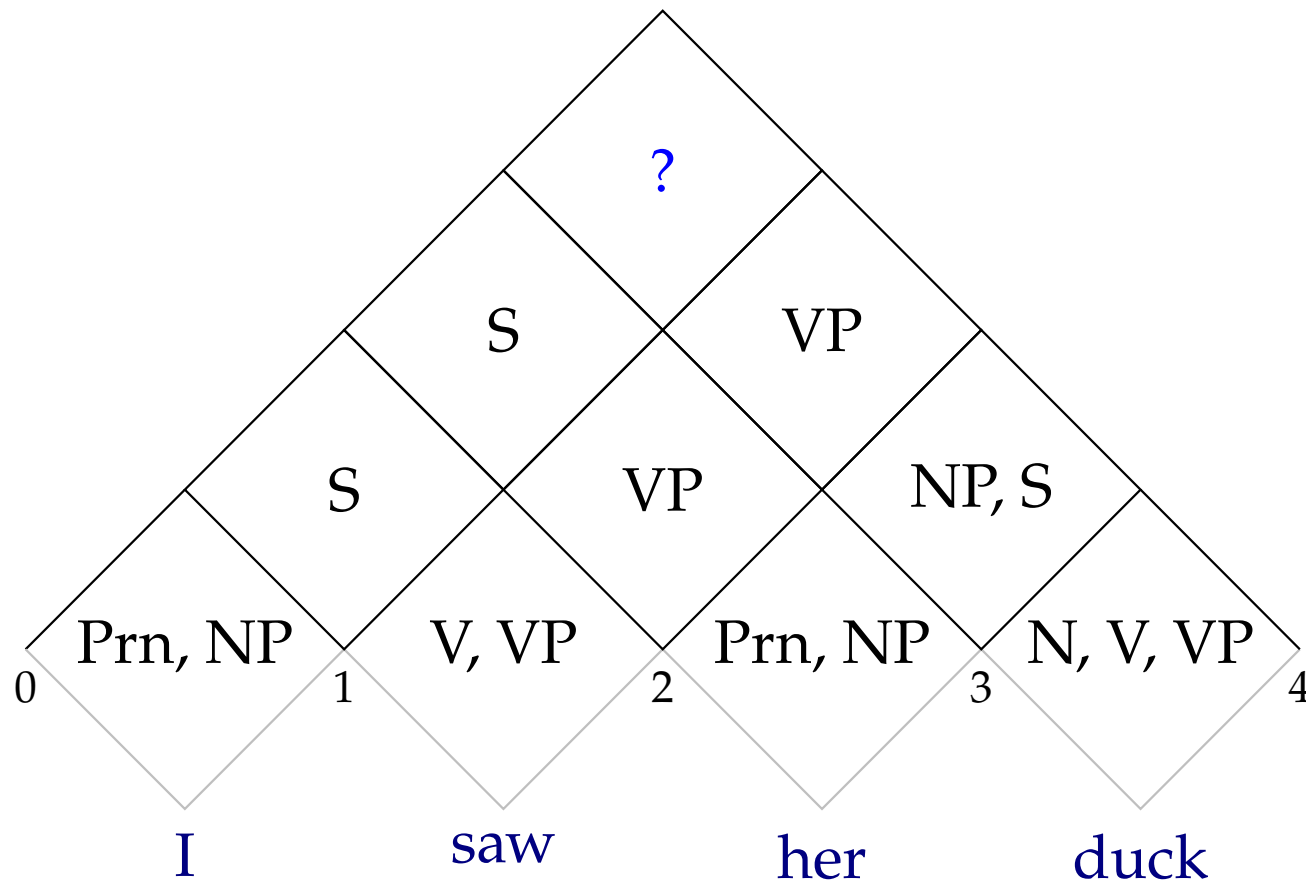
S $\rightarrow$ NP VP
 S $\rightarrow$ Aux NP VP
 NP $\rightarrow$ Det N
 NP $\rightarrow$ Prn
 NP $\rightarrow$ NP PP
 VP $\rightarrow$ V NP
 VP $\rightarrow$ V
 VP $\rightarrow$ VP PP
 PP $\rightarrow$ Prp NP
 N $\rightarrow$ duck
 N $\rightarrow$ park
 V $\rightarrow$ duck
 V $\rightarrow$ ducks
 V $\rightarrow$ saw
 Prn $\rightarrow$ she | her
 Prp $\rightarrow$ in | with
 Det $\rightarrow$ a | the



$S \rightarrow NP VP$
 $S \rightarrow Aux NP VP$
 $NP \rightarrow Det N$
 $NP \rightarrow Prn$
 $NP \rightarrow NP PP$
 $VP \rightarrow V NP$
 $VP \rightarrow V$
 $VP \rightarrow VP PP$
 $PP \rightarrow Prp NP$
 $N \rightarrow duck$
 $N \rightarrow park$
 $V \rightarrow duck$
 $V \rightarrow ducks$
 $V \rightarrow saw$
 $Prn \rightarrow she | her$
 $Prp \rightarrow in | with$
 $Det \rightarrow a | the$

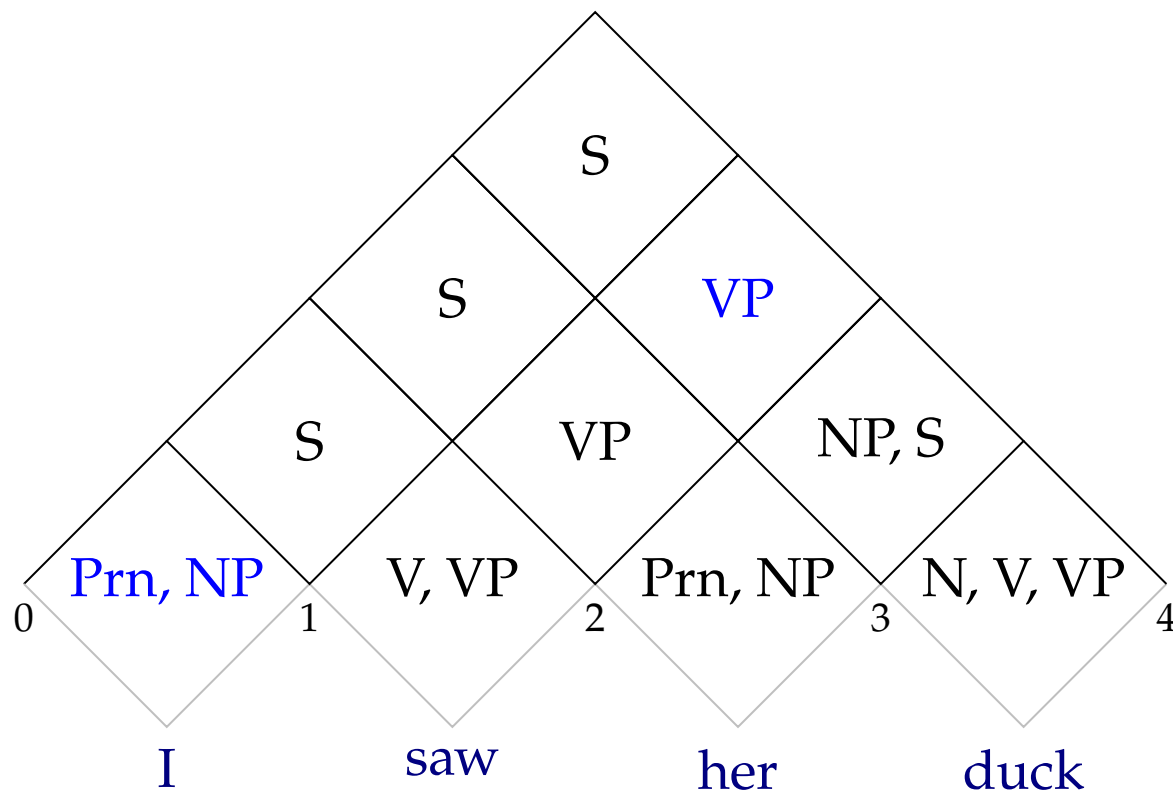


- S $\rightarrow$ NP VP
- S $\rightarrow$ Aux NP VP
- NP $\rightarrow$ Det N
- NP $\rightarrow$ Prn
- NP $\rightarrow$ NP PP
- VP $\rightarrow$ V NP
- VP $\rightarrow$ V
- VP $\rightarrow$ VP PP
- PP $\rightarrow$ Prp NP
- N $\rightarrow$ duck
- N $\rightarrow$ park
- V $\rightarrow$ duck
- V $\rightarrow$ ducks
- V $\rightarrow$ saw
- Prn $\rightarrow$ she | her
- Prp $\rightarrow$ in | with
- Det $\rightarrow$ a | the

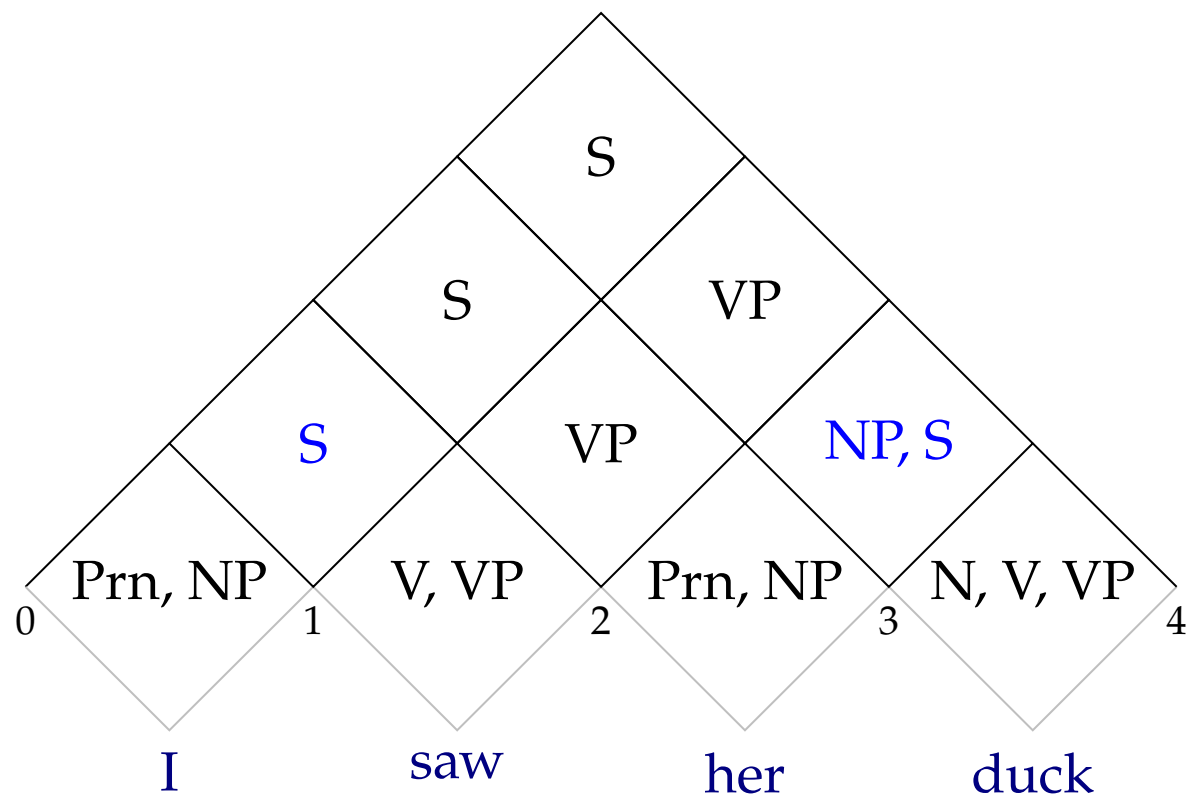


- S → NP VP
- S → Aux NP VP
- NP → Det N
- NP → Prn
- NP → NP PP
- VP → V NP
- VP → V
- VP → VP PP
- PP → Prp NP
- N → duck
- N → park
- V → duck
- V → ducks
- V → saw
- Prn → she | her
- Prp → in | with
- Det → a | the

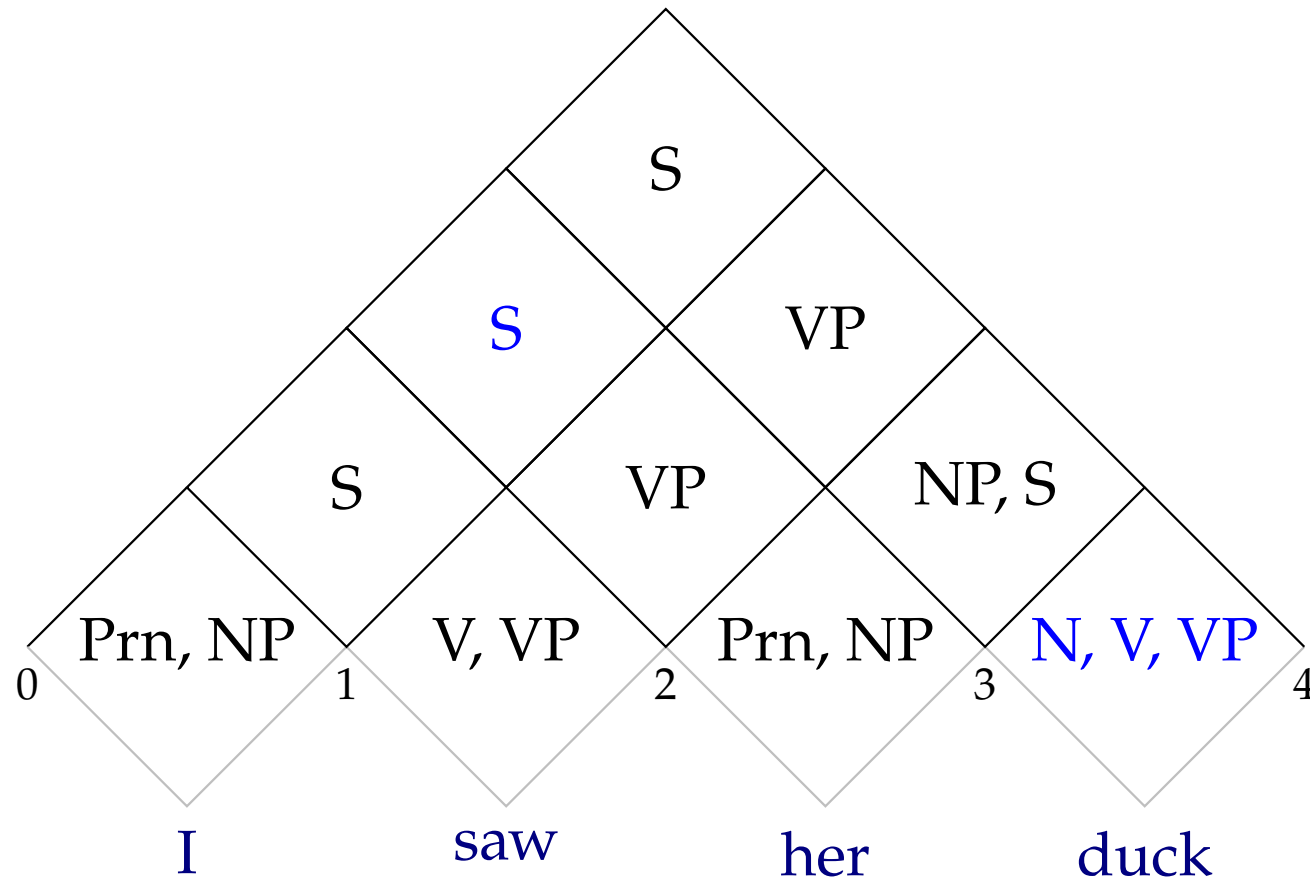
$S \rightarrow NP VP$



- $S \rightarrow NP VP$
- $S \rightarrow Aux NP VP$
- $NP \rightarrow Det N$
- $NP \rightarrow Prn$
- $NP \rightarrow NP PP$
- $VP \rightarrow V NP$
- $VP \rightarrow V$
- $VP \rightarrow VP PP$
- $PP \rightarrow Prp NP$
- $N \rightarrow duck$
- $N \rightarrow park$
- $V \rightarrow duck$
- $V \rightarrow ducks$
- $V \rightarrow saw$
- $Prn \rightarrow she \mid her$
- $Prp \rightarrow in \mid with$
- $Det \rightarrow a \mid the$



S → NP VP
 S → Aux NP VP
 NP → Det N
 NP → Prn
 NP → NP PP
 VP → V NP
 VP → V
 VP → VP PP
 PP → Prp NP
 N → duck
 N → park
 V → duck
 V → ducks
 V → saw
 Prn → she | her
 Prp → in | with
 Det → a | the



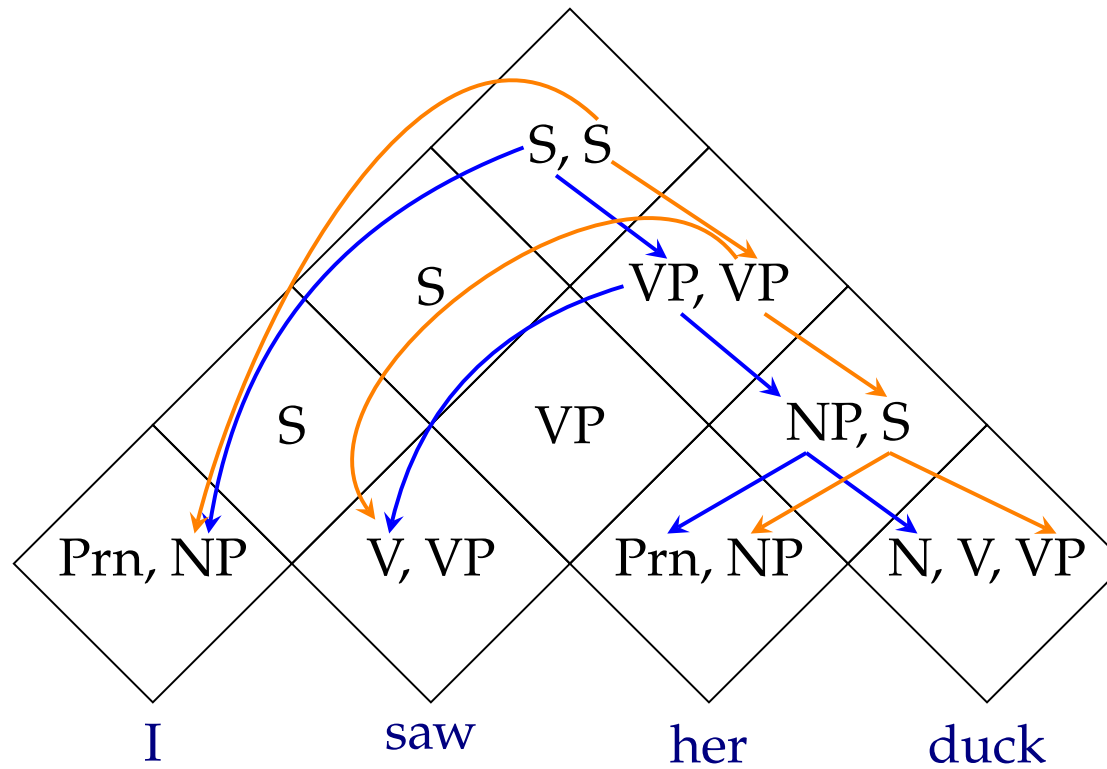
- S → NP VP
- S → Aux NP VP
- NP → Det N
- NP → Prn
- NP → NP PP
- VP → V NP
- VP → V
- VP → VP PP
- PP → Prp NP
- N → duck
- N → park
- V → duck
- V → ducks
- V → saw
- Prn → she | her
- Prp → in | with
- Det → a | the

NP, Prn	S	S	S
	V, VP	VP	VP
		Prn	NP, S
			V, N, NP
0 she	1 saw	2 her	3 duck 4

Parsing vs. recognition

- We went through a recognition example
- Recognition accepts or rejects a sentence based on a grammar
- For parsing, we want to know the derivations that yielded a correct parse
- To recover parse trees, we
 - we follow the same procedure as recognition
 - add back links to keep track of the derivations

CKY Parsing



CKY summary

- CKY avoids re-computing the analyses by storing the earlier analyses (of sub-spans) in a table
- CKY requires the grammar to be in CNF
- CKY has $O(n^3)$ recognition complexity
- For parsing we need to keep track of backlinks
- CKY can efficiently store all possible parses in a chart

Neural Parsing

Traditional parsing relied on PCFG (Probabilistic Context-Free Grammars), where humans defined rules and the computer calculated the likelihood of each rule. Neural parsing changes this.

- Traditional approach: Uses probabilistic context-free grammars (PCFGs) and dynamic programming (e.g., CKY algorithm).
- Neural shift: Modern approaches replace rule-based scoring with neural span scorers, enabling more flexible, data-driven parsing.
- Why Span-Based?
- Span-based parsing: Focuses on evaluating the likelihood of word spans forming valid constituents, instead of sequential transitions or grammar rules.
- Advantage: Direct scoring over spans harnesses context, improving accuracy and robustness over hand-crafted grammars.

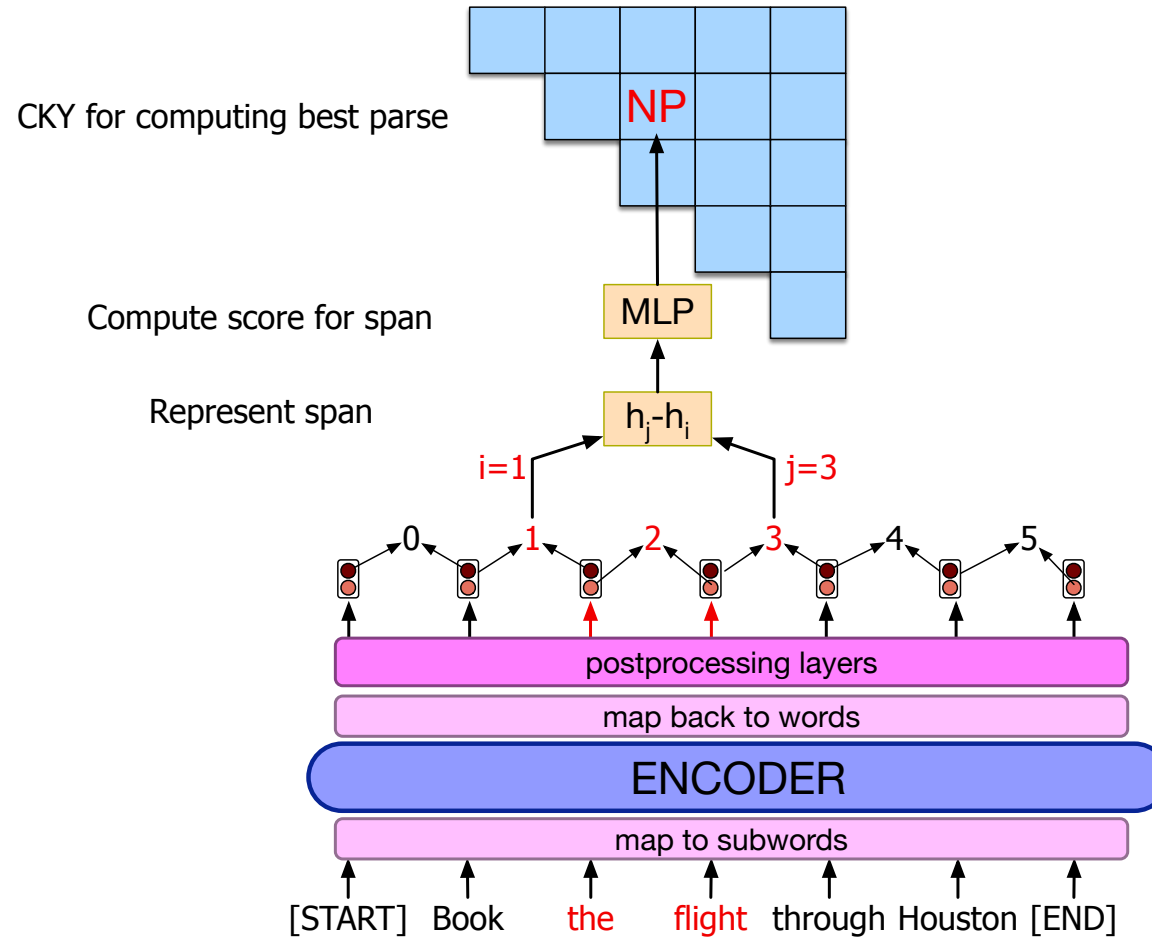
Neural Span Scoring

- For each start–end span (i,j) , a neural network computes a score $s(i,j,l)$ for possible constituent label l over that span.
- Input representations: Typically use contextual embeddings (from BiLSTM or Transformer, like BERT) for each position.
- Output: Constituent scores are assigned for all possible labeled spans.

Computing Scores for a Span

- Let's begin by considering just the constituent (we'll call it a span) that lies between fencepost positions i and j with non-terminal symbol label l .
- We'll build a system to assign a score $s(i,j,l)$ to this constituent span.
- The input word tokens are embedded by passing them through a pretrained language model like BERT
- We'll first need to convert the BERT outputs to word representations.
- One standard way of doing this is to simply use the first subword unit as the representation for the entire word; using the last subword unit, or the sum of all the subword units are also common.

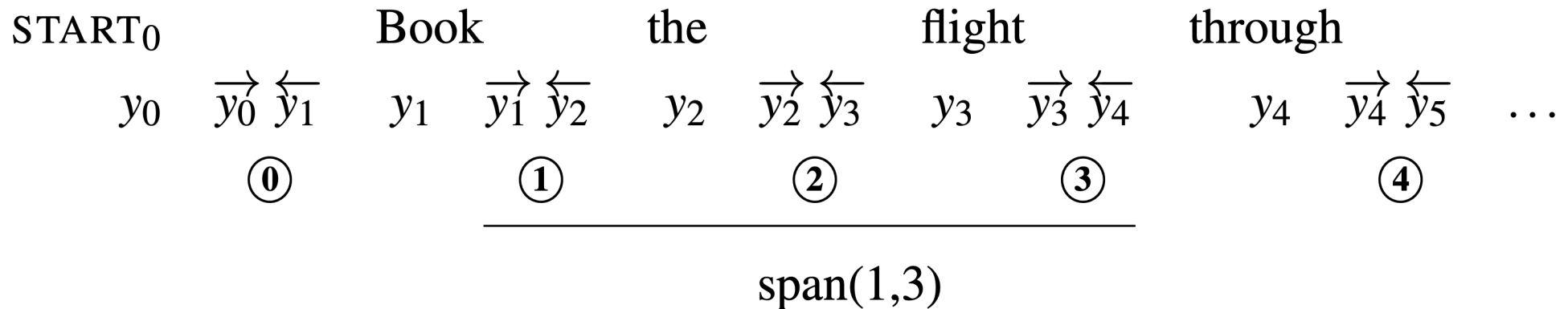
Span Representation Techniques



Span Representation Techniques

- The embeddings can then be passed through some postprocessing layers for example, use 8 Transformer layers.
- The resulting word encoder outputs y_t are then used to compute a span score.
- First, we must map the word encodings (indexed by word positions) to span encodings (indexed by fenceposts).
- We do this by representing each fencepost with two separate values; the intuition is that a span endpoint to the right of a word represents different information than a span endpoint to the left of a word.

- We convert each word output y_t into a (leftward-pointing) value for spans ending at this fencepost, $\overleftarrow{y}_t$, and a (rightward-pointing) value $\overrightarrow{y}_t$ for spans beginning at this fencepost, by splitting y_t into two halves.
- representation of *the flight*



- A traditional way to represent a span is to take the difference between the embeddings of its start and end
- representing span (i,j) by subtracting the embedding of i from the embedding of j.
- Here we represent a span by concatenating the difference of each of its fencepost components:

$$v(i, j) = [\overrightarrow{y_j} - \overrightarrow{y_i} ; \overleftarrow{y_{j+1}} - \overleftarrow{y_{i+1}}]$$

- The span vector $\mathbf{v}$ is then passed through an MLP span classifier, with two fully-connected layers and one ReLU activation function, whose output dimensionality is the number of possible non-terminal labels:

$$s(i, j, \cdot) = \mathbf{W}_2 \text{ReLU}(\text{LayerNorm}(\mathbf{W}_1 \mathbf{v}(i, j)))$$

- The MLP then outputs a score for each possible non-terminal.

Integrating Span Scores into a Parse

- Now we have a score for each labeled constituent span $s(i,j,l)$.
- But we need a score for an entire parse tree.
- Formally a tree T is represented as a set of $|T|$ such labelled spans, with the t^{th} span starting at position i_t and ending at position j_t , with label l_t :
- $T = \{(i_t, j_t, l_t) : t = 1, \dots, |T|\}$
- Thus once we have a score for each span, the parser can compute a score for the whole tree $s(T)$ simply by summing over the scores of its constituent spans:

$$s(T) = \sum_{(i,j,l) \in T} s(i, j, l)$$

- And we can choose the final parse tree as the tree with the maximum score:

$$\hat{T} = \operatorname{argmax}_T s(T)$$

- The simplest method to produce the most likely parse is to greedily choose the highest scoring label for each span.
- This greedy method is not guaranteed to produce a tree, since the best label for a span might not fit into a complete tree.
- In practice, however, the greedy method tends to find trees; in their experiments Gaddy et al. (2018) finds that 95% of predicted bracketings form valid trees.
- Nonetheless it is more common to use a variant of the CKY algorithm to find the full parse.

Evaluating Parsers

- The standard tool for evaluating parsers that assign a single parse tree to a sentence is the PARSEVAL metrics (Black et al., 1991).
- The PARSEVAL metric measures how much the constituents in the hypothesis parse tree look like the constituents in a hand-labeled, reference parse.
- PARSEVAL thus requires a human-labeled reference (or “gold standard”) parse tree for each sentence in the test set
- we generally draw these reference parses from a treebank like the Penn Treebank.

- A constituent in a hypothesis parse C_h of a sentence s is labeled correct if there is a constituent in the reference parse C_r with the same starting point, ending point, and non-terminal symbol.

$$\text{labeled recall:} = \frac{\text{\# of correct constituents in hypothesis parse of } s}{\text{\# of total constituents in reference parse of } s}$$

$$\text{labeled precision:} = \frac{\text{\# of correct constituents in hypothesis parse of } s}{\text{\# of total constituents in hypothesis parse of } s}$$

Benefits and Limitations

- Benefits:
 - Flexibility in constituent selection (no strict grammar constraints)
 - State-of-the-art accuracy
- Limitations:
 - May require large annotated datasets
 - Sensitive to representation and decoding choices